

UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL MENDOZA

Tecnicatura Universitaria en Programación

**PLATAFORMA DISTRIBUIDA PARA EL MONITOREO DE
INTEGRIDAD DE ARCHIVOS EN SISTEMAS LINUX CON
MOTOR DE DECISIÓN BASADO EN REGLAS, VALIDACIÓN
HUMANA Y ORQUESTACIÓN DE NOTIFICACIONES**

Trabajo Final de carrera presentado para optar al título de Técnico Universitario en Programación

Autores

Ezequiel González

Nicolás Castro

Director de la carrera

Alberto Cortez

Mendoza, Argentina — 2026

DECLARACIÓN DE ORIGINALIDAD

Los autores, Ezequiel González y Nicolás Castro, declaran bajo juramento que el presente Trabajo Final de carrera es producto de su labor personal y no ha sido presentado previamente para obtener otro título académico en esta ni en otra institución educativa. Las fuentes bibliográficas, estándares técnicos y publicaciones consultadas han sido citadas conforme al sistema de referenciación APA en su séptima edición. Los diagramas arquitectónicos, los esquemas de la base de datos, las decisiones de diseño y los artefactos de documentación producidos son originales, o bien incorporan componentes de terceros que se reconocen explícitamente y que se emplean bajo licencias compatibles con el presente trabajo.

Durante la fase de redacción se utilizaron herramientas de inteligencia artificial generativa con el único propósito de la corrección estilística, la verificación de consistencia y la organización expositiva del material. El contenido técnico, las decisiones de diseño arquitectónico, la selección del stack tecnológico, la formulación del protocolo experimental y las conclusiones son de autoría exclusiva de los firmantes.

Mendoza, Argentina — 2026.

Ezequiel González

Nicolás Castro

DEDICATORIA

A nuestras familias, por el sostén silencioso durante los años de formación.

A los docentes de la Tecnicatura Universitaria en Programación de la UTN-FRM, cuyas explicaciones pacientes dieron forma a la vocación técnica que guía este trabajo.

A las organizaciones que, con recursos acotados, sostienen a diario la protección de información crítica con las herramientas que tienen a mano.

AGRADECIMIENTOS

Los autores desean expresar su reconocimiento al cuerpo docente y no docente de la Tecnicatura Universitaria en Programación de la Facultad Regional Mendoza de la Universidad Tecnológica Nacional, cuyo acompañamiento pedagógico durante los años de cursada hizo posible la culminación del presente trabajo. Se agradece de manera especial al director de la carrera, Alberto Cortez, por sus orientaciones académicas durante la formulación y el desarrollo del proyecto.

Se agradece, asimismo, a la comunidad global de software libre, cuyas aportaciones desinteresadas constituyen el fundamento técnico sobre el cual se asienta la propuesta aquí desarrollada. Resulta particularmente relevante la mención a los mantenedores de los proyectos FastAPI, SQLAlchemy, PostgreSQL, Valkey, pyfanotify y n8n, sin cuya existencia la presente tesis no habría sido posible. Finalmente, los autores agradecen a sus familias y allegados por la paciencia demostrada y por el aliento sostenido durante todo el proceso.

RESUMEN

El presente Trabajo Final de carrera aborda el diseño de una plataforma distribuida destinada al Monitoreo de Integridad de Archivos en sistemas operativos Linux. La propuesta responde a una brecha documentada en el panorama contemporáneo: existe una distancia significativa entre las soluciones comerciales del segmento, frecuentemente costosas y opacas

para el usuario final, y las alternativas artesanales basadas en la verificación periódica mediante scripts, cuyas limitaciones estructurales comprenden ventanas temporales de ceguera propias del encuestamiento, almacenamiento local de la baseline expuesto a manipulación y ausencia de integración con los canales operativos de respuesta a incidentes.

La arquitectura propuesta articula seis componentes desacoplados que se comunican mediante mensajería asíncrona. Un agente desarrollado en lenguaje Python opera como servicio nativo del sistema en cada anfitrión monitoreado, emplea el subsistema fanotify del núcleo Linux para la detección reactiva de eventos de archivo y ejecuta localmente un motor de decisión con cuatro niveles de acción. Un backend construido sobre FastAPI con base de datos PostgreSQL se organiza en capas bajo principios de arquitectura limpia, adoptando los patrones de Unit of Work y Repository. Una capa de mensajería basada en Valkey Streams transporta los eventos del agente hacia el backend y los comandos del backend hacia el agente. Un motor de orquestación n8n enruta las notificaciones externas, con mecanismos de reintento y cola de mensajes fallidos. Una aplicación frontend desarrollada en React habilita la validación humana de los eventos y la administración del sistema. Un servidor central contenedorizado con Docker Compose integra estos servicios, a los cuales los agentes nativos se conectan mediante autenticación mutua de certificados TLS.

Entre las decisiones técnicas destacadas figuran el cifrado autenticado de la baseline en reposo mediante AES-256-GCM, con derivación de clave a cargo de la función HKDF-SHA256; la firma HMAC-SHA256 de los comandos emitidos por el backend; un protocolo de entrega con garantía exactamente-una-vez implementado mediante confirmación bidireccional sobre los streams de Valkey; la resolución de concurrencia en el flujo de aprobación humana a través de bloqueo optimista y cadena de eventos superados; un mecanismo de detección de desincronización horaria con umbral de cinco minutos como defensa anti-replay; y la integración con notificaciones mediante fallbacks automáticos hacia canales alternativos cuando el orquestador principal resulta indisponible.

La propuesta se alinea con los requerimientos de la Ley 25.326 de Protección de Datos Personales, con las Resoluciones 47/2018 y 126/2024 de la Agencia de Acceso a la Información Pública, con la Resolución 44/2023 que aprueba la Segunda Estrategia Nacional de Ciberseguridad, y con el Decreto de Necesidad y Urgencia 941/2025 que creó el Centro Nacional de Ciberseguridad como autoridad nacional en la materia.

Al momento de la presentación, el proyecto se encuentra en fase de documentación y diseño detallado. El protocolo experimental y las planillas de captura reproducibles se documentan en el Capítulo 5 con el propósito expreso de habilitar la replicación por parte de terceros.

Palabras clave: *Monitoreo de Integridad de Archivos, fanotify, SHA-256, FastAPI, PostgreSQL, Valkey, n8n, SOAR, Argon2id, autenticación mutua, Zero Trust, Ley 25.326, ciberseguridad en Argentina, Linux.*

ABSTRACT

This Final Project addresses the design of a distributed File Integrity Monitoring (FIM) platform for Linux operating systems. The proposal responds to a documented gap between commercial FIM solutions, which are expensive and opaque to end users, and handcrafted alternatives based on polling scripts whose structural weaknesses include temporal blind spots, locally stored baselines exposed to tampering, and the absence of integration with operational incident response channels.

The proposed architecture articulates six decoupled components that communicate via asynchronous messaging. A Python agent runs as a native system service on each monitored host, using the Linux kernel fanotify subsystem for reactive event detection and executing a local decision engine with four action levels. A FastAPI backend with a PostgreSQL database is organized in layers under clean architecture principles, adopting Unit of Work and Repository patterns. A Valkey Streams messaging layer carries events from agent to backend and commands from backend to agent. An n8n orchestration engine routes external notifications with retry and dead-letter mechanisms. A React frontend enables human validation of events and administration. A centralized server containerized with Docker Compose integrates these services, to which the native agents connect via mutual TLS authentication.

Key technical decisions include authenticated encryption of the baseline at rest using AES-256-GCM with HKDF-SHA256 key derivation, HMAC-SHA256 signed commands, exactly-once delivery via bidirectional acknowledgement over Valkey Streams, optimistic

concurrency control on the human approval workflow with superseded event chaining, clock-skew replay detection with a five-minute threshold, and automatic fallback to alternative notification channels upon primary orchestrator unavailability.

The proposal aligns with Argentine regulatory requirements: Law 25.326, AAIP Resolutions 47/2018 and 126/2024, Resolution 44/2023 establishing the Second National Cybersecurity Strategy, and DNU 941/2025 establishing the National Cybersecurity Center. At submission time, the project is in documentation and detailed design phase; experimental protocol and data capture sheets are documented in Chapter 5 to enable third-party replication.

Keywords: *File Integrity Monitoring, fanotify, SHA-256, FastAPI, PostgreSQL, Valkey, n8n, SOAR, Argon2id, mutual TLS, Zero Trust, Argentine Law 25.326, Linux.*

ÍNDICE GENERAL

DECLARACIÓN DE ORIGINALIDAD	1
DEDICATORIA	1
AGRADECIMIENTOS	2
RESUMEN	2
ABSTRACT	4
ÍNDICE GENERAL	5
ÍNDICE DE TABLAS	5
ÍNDICE DE FIGURAS	6
LISTA DE ABREVIATURAS Y SIGLAS	6
CAPÍTULO 1. INTRODUCCIÓN	1
1.1 Contexto del problema	1
1.2 Planteamiento del problema	2
1.3 Justificación	4
1.4 Objetivo general	4
1.5 Objetivos específicos	5
1.6 Hipótesis y criterios de aceptación	6
1.7 Alcance y limitaciones	6
CAPÍTULO 2. MARCO TEÓRICO	8
2.1 Seguridad informática y la tríada CIA	8
2.2 Funciones hash criptográficas	9
2.3 Monitoreo de Integridad de Archivos	9

2.4 Marco normativo argentino aplicable	10
2.5 Estado del arte de soluciones FIM	12
2.6 Subsistemas del núcleo Linux para detección de eventos de archivo	12
2.7 Paradigma SOAR y orquestación con n8n	13
2.8 Stack backend: FastAPI, SQLAlchemy y PostgreSQL	14
2.9 Mensajería con Valkey Streams	15
2.10 Primitivas criptográficas adoptadas	15
2.11 Zero Trust y raíz de confianza	16
2.12 Modelo de amenazas STRIDE	17
2.13 Patrones de diseño de software aplicados	18
CAPÍTULO 3. MARCO METODOLÓGICO	19
3.1 Tipo de investigación	19
3.2 Enfoque metodológico	19
3.3 Diseño experimental con grupo de control	20
3.4 Variables e indicadores	21
3.5 Herramientas utilizadas	21
3.6 Técnicas de recolección	22
3.7 Protocolo experimental	23
CAPÍTULO 4. DESARROLLO DE LA INVESTIGACIÓN	25
4.1 Arquitectura general y distribución física	25
4.2 Esquema de persistencia	28
4.3 Flujo del agente de monitoreo	30
4.4 Motor de decisión, cadena superseded y protocolo ACK	33
4.5 Flujo del backend: lifecycle formal, bloqueo optimista y anti-replay	35
4.6 Frontend administrativo y hardening web	37
4.7 Orquestación de notificaciones con fallbacks automáticos	38
4.8 Seguridad criptográfica del sistema	40
4.9 Organización del código: arquitectura por capas	42
4.10 Contenedorización del servidor central y despliegue del agente	43
4.11 Operaciones y observabilidad	44
4.12 Estado real del proyecto	45
CAPÍTULO 5. RESULTADOS	46
5.1 Consideración metodológica sobre la presentación de resultados	46
5.2 Protocolo de medición de latencia de detección	47
5.3 Protocolo de medición de tiempo de notificación	47
5.4 Protocolo de cobertura de automatización del triage	48
5.5 Validación funcional mediante casos de prueba	49
5.6 Protocolo de resiliencia offline	49
5.7 Comparación con grupo de control	50
5.8 Síntesis de criterios de aceptación	50
CAPÍTULO 6. DISCUSIÓN	52
6.1 Interpretación de los resultados esperados	52
6.2 Contribuciones diferenciales del diseño	52
6.3 Adecuación al marco normativo argentino	53

6.4 Amenazas a la validez interna	53
6.5 Amenazas a la validez externa	54
6.6 Limitaciones tecnológicas del stack adoptado	54
6.7 Posicionamiento respecto de Falco y Tetragon	55
CAPÍTULO 7. CONCLUSIONES	57
7.1 Cumplimiento de los objetivos formulados	57
7.2 Contribución teórica	57
7.3 Contribución metodológica	58
7.4 Contribución práctica y normativa	58
7.5 Contribución formativa	58
7.6 Reconocimiento de limitaciones y compromiso de adenda	58
CAPÍTULO 8. RECOMENDACIONES Y TRABAJO FUTURO	60
8.1 Completar la fase de implementación y validación	60
8.2 Alta disponibilidad del backend	60
8.3 Extensión de la raíz de confianza al núcleo	60
8.4 Arquitectura híbrida con Tetragon	60
8.5 Respuesta automatizada activa	61
8.6 Gestión inteligente de falsos positivos	61
8.7 Integración con ecosistemas SIEM	61
8.8 Empaquetado nativo y recomendaciones operativas	61
CAPÍTULO 9. CONSIDERACIONES ÉTICAS	63
9.1 Principios orientadores del diseño	63
9.2 Responsabilidad en la divulgación técnica	63
9.3 Uso de herramientas de inteligencia artificial	64
9.4 Adecuación a la normativa argentina	64
CAPÍTULO 10. GLOSARIO	65
10.1 Términos técnicos	65
10.2 Instrumentos legales argentinos invocados	69
CAPÍTULO 11. REFERENCIAS BIBLIOGRÁFICAS	72
11.1 Normativa argentina	72
11.2 Estándares internacionales	73
11.3 Reportes de amenazas y estado de la industria (2024-2025)	73
11.4 Publicaciones académicas recientes	74
11.5 Libros y manuales metodológicos	74
11.6 Documentación técnica oficial	74
CAPÍTULO 12. ANEXOS	76
Anexo A. Flujo de procesamiento del agente	76
Anexo B. Flujo de aprobación humana en el backend	77
Anexo C. Extracto del esquema de persistencia	78
Anexo D. Estructura del archivo Docker Compose	78
Anexo E. Unit file de systemd del agente	79
Anexo F. Disponibilidad del código y compromiso de adenda	79

ÍNDICE DE TABLAS

- Tabla 1. Criterios de aceptación preestablecidos
- Tabla 2. Modelo STRIDE aplicado al sistema FIM
- Tabla 3. Variables e indicadores experimentales
- Tabla 4. Stack tecnológico por componente
- Tabla 5. Tablas principales del esquema de persistencia
- Tabla 6. Políticas de resiliencia del agente
- Tabla 7. Máquina de estados explícita del evento
- Tabla 8. Orden de precedencia para entrega de notificaciones
- Tabla 9. Distribución de controles criptográficos
- Tabla 10. Estado actual del proyecto por componente
- Tabla 11. Planilla de captura — Latencia de detección
- Tabla 12. Planilla de captura — Tiempo de notificación
- Tabla 13. Planilla de captura — Cobertura de automatización del triage
- Tabla 14. Planilla de captura — Resiliencia offline
- Tabla 15. Planilla de captura — Comparación experimental vs. control
- Tabla 16. Síntesis de criterios de aceptación a completar

ÍNDICE DE FIGURAS

- Figura 1. Arquitectura general del sistema y distribución física
- Figura 2. Modelo entidad-relación de las tablas principales de PostgreSQL
- Figura 3. Flujo de procesamiento del agente: tres fases de operación

Figura 4. Diagrama de secuencia del protocolo de entrega exactamente-una-vez

Figura 5. Máquina de estados explícita del evento (siete estados posibles)

Figura 6. NotificationDispatcher: cascada de fallbacks automáticos

LISTA DE ABREVIATURAS Y SIGLAS

El listado siguiente reúne las abreviaturas y siglas que el lector encontrará a lo largo del documento. Su inclusión busca facilitar la lectura sin necesidad de regresar al Glosario del Capítulo 10, que contiene las definiciones extendidas y el contexto de cada término.

Tabla A. Abreviaturas y siglas empleadas en el documento

Sigla	Expansión / significado
AAIP	Agencia de Acceso a la Información Pública (autoridad nacional argentina)
AES-GCM	Advanced Encryption Standard en modo Galois/Counter Mode (cifrado autenticado)
API	Application Programming Interface (interfaz de programación de aplicaciones)
APA	American Psychological Association (estilo de citación bibliográfica, séptima edición)
ASGI	Asynchronous Server Gateway Interface (protocolo web asíncrono para Python)
CA	Certification Authority (autoridad certificadora)
CIA	Confidentiality, Integrity, Availability (triada fundamental de seguridad informática)
CNC	Centro Nacional de Ciberseguridad (autoridad nacional en Argentina)
CONEAU	Comisión Nacional de Evaluación y Acreditación Universitaria
CSP	Content Security Policy (política de seguridad de contenido web)
CSR	Certificate Signing Request (solicitud de firma de certificado)
DA	Decisión Administrativa (instrumento normativo de la Jefatura de Gabinete)
DLQ	Dead Letter Queue (cola de mensajes fallidos)
DNU	Decreto de Necesidad y Urgencia (instrumento normativo del Poder Ejecutivo argentino)

Sigla	Expansión / significado
eBPF	extended Berkeley Packet Filter (tecnología de extensibilidad del núcleo Linux)
FIM	File Integrity Monitoring (Monitoreo de Integridad de Archivos)
FIPS	Federal Information Processing Standards (estándares federales de los EE. UU.)
HKDF	HMAC-based Key Derivation Function (función de derivación de claves, RFC 5869)
HMAC	Hash-based Message Authentication Code (autenticación de mensajes basada en hash)
HTTP / HTTPS	HyperText Transfer Protocol (sobre TLS en su versión segura)
IMA	Integrity Measurement Architecture (subsistema del núcleo Linux)
JSON	JavaScript Object Notation (formato de intercambio de datos)
JWT	JSON Web Token (formato de token de autenticación)
MTD / MTR	Mean Time to Detect / Mean Time to Respond (tiempos medios de detección y respuesta)
mTLS	mutual Transport Layer Security (autenticación mutua mediante certificados TLS)
NIST	National Institute of Standards and Technology (de los Estados Unidos)
NFS	Network File System (sistema de archivos distribuido)
ORM	Object-Relational Mapper (mapeador objeto-relacional)
PKI	Public Key Infrastructure (infraestructura de clave pública)
RFC	Request For Comments (publicación técnica del IETF)
SHA-256	Secure Hash Algorithm de 256 bits (FIPS 180-4)
SIEM	Security Information and Event Management
SOAR	Security Orchestration, Automation and Response
SSE	Server-Sent Events (flujo de eventos del servidor)
STRIDE	Marco para modelado de amenazas (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege)
TLS	Transport Layer Security (protocolo de cifrado de canal)
TTL	Time To Live (tiempo de vida de una entrada en caché)
UoW	Unit of Work (patrón de diseño para transacciones)

Sigla	Expansión / significado
UTN-FRM	Universidad Tecnológica Nacional — Facultad Regional Mendoza
UUID	Universally Unique Identifier (identificador único universal)

CAPÍTULO 1. INTRODUCCIÓN

«La integridad de los datos no es un estado que se alcanza una vez: es un proceso que debe sostenerse todos los días, a cada hora, a cada segundo.»

— Adaptado de Menezes, van Oorschot y Vanstone (1996)

El presente capítulo introduce al lector en la problemática que motiva la realización del Trabajo Final, delimita el objeto de estudio, explicita los objetivos general y específicos, formula la hipótesis de trabajo con sus criterios de aceptación preestablecidos, y establece el alcance junto con las limitaciones del proyecto. La exposición parte de una descripción general del panorama contemporáneo de la ciberseguridad organizacional y profundiza progresivamente hacia el problema específico abordado, siguiendo la lógica del embudo característica de la introducción académica.

1.1 Contexto del problema

La transformación digital ha convertido a la ciberseguridad en una disciplina de relevancia estratégica tanto para organizaciones privadas como para entidades gubernamentales. En el marco conceptual clásico, la tríada conocida como CIA sintetiza los tres pilares fundamentales que todo sistema de protección de la información debe preservar: confidencialidad, integridad y disponibilidad. La integridad, propiedad central del presente trabajo, refiere a la garantía de que los datos no han sido modificados de forma no autorizada respecto de un estado de referencia legítimo. En el contexto argentino, esta propiedad se encuentra consagrada normativamente en el artículo 9.º de la Ley 25.326 de Protección de Datos Personales, que exige adoptar medidas técnicas y organizativas tendientes a evitar la adulteración, la pérdida, la consulta o el tratamiento no autorizado de los datos, y que permitan, adicionalmente, detectar desviaciones de información (Honorable Congreso de la Nación Argentina, 2000).

Los reportes de amenazas publicados durante el bienio 2024-2025 coinciden en señalar que las técnicas de persistencia basadas en la modificación encubierta de archivos del sistema continúan entre los vectores de compromiso más frecuentes. Un caso paradigmático que ilustra la severidad de las modificaciones maliciosas es el incidente CVE-2024-3094 sobre la biblioteca de compresión XZ Utils, en el que una puerta trasera fue insertada

mediante alteración progresiva del repositorio oficial y detectada por casualidad a partir de una anomalía menor de rendimiento detectada por un ingeniero ajeno al proyecto (Clément et al., 2025). El episodio puso en evidencia tanto la dificultad de detectar modificaciones sofisticadas como el valor forense del monitoreo granular de archivos del sistema, especialmente cuando se dispone de un historial criptográfico auditable de cada cambio.

En el ámbito argentino, el escenario se ha visto reforzado por una actividad regulatoria intensa en los años recientes. La Resolución 44/2023 de la Secretaría de Innovación Pública, que aprueba la Segunda Estrategia Nacional de Ciberseguridad, establece como objetivos centrales la protección de infraestructuras críticas nacionales y el fortalecimiento de las capacidades de prevención, detección y respuesta (Secretaría de Innovación Pública, 2023). Posteriormente, el Decreto de Necesidad y Urgencia 941/2025, publicado el 2 de enero de 2026, creó el Centro Nacional de Ciberseguridad como organismo descentralizado con rango de autoridad nacional en la materia, separando formalmente las funciones de ciberseguridad de las de ciberinteligencia (Poder Ejecutivo Nacional, 2025). La Resolución 47/2018 de la Agencia de Acceso a la Información Pública, por su parte, establece las medidas de seguridad recomendadas para el tratamiento de datos personales en medios informatizados, organizadas en cinco procesos: recolección, control de acceso, conservación, destrucción y gestión de incidentes (AAIP, 2018).

A pesar de la relevancia normativa y técnica, las soluciones comerciales de Monitoreo de Integridad de Archivos disponibles en el mercado presentan una combinación de características que dificulta su adopción por organizaciones de porte medio. Entre ellas, el costo elevado del licenciamiento anual, la opacidad arquitectónica propia del software propietario que impide auditar el comportamiento interno del sistema de seguridad, y la fricción de integración con los flujos de trabajo preexistentes en la organización. En consecuencia, una proporción significativa de organizaciones opta por alternativas artesanales basadas en scripts de verificación programada, cuyas limitaciones estructurales se analizan en el apartado siguiente.

1.2 Planteamiento del problema

A partir del diagnóstico precedente se identifica una brecha significativa entre las capacidades que ofrecen las soluciones comerciales del segmento y las posibilidades efectivas de adopción por parte de organizaciones con recursos limitados. Las alternativas artesanales

habitualmente implementadas, típicamente constituidas por scripts que recorren periódicamente un conjunto de rutas calculando sus huellas digitales y comparándolas contra un archivo de referencia local, presentan tres limitaciones estructurales de orden crítico que conviene caracterizar con detalle.

La primera limitación, de naturaleza temporal, reside en que estos scripts operan bajo un modelo de encuestamiento: se ejecutan a intervalos programados que típicamente oscilan entre los quince minutos y la hora. Entre dos ejecuciones consecutivas, cualquier modificación maliciosa introducida y posteriormente revertida permanece por completo fuera del radar de detección. Esta ventana de ceguera se traduce, en el mejor de los casos, en un Tiempo Medio de Detección equivalente a la mitad del intervalo de encuestamiento, y es conceptualmente imposible de cerrar sin cambiar el modelo subyacente hacia uno reactivo basado en eventos del sistema operativo.

La segunda limitación, de naturaleza arquitectónica, consiste en que estos scripts almacenan la baseline en texto claro en el mismo sistema que monitorean, exponiéndola así a manipulación directa en caso de compromiso. Un atacante con privilegios administrativos puede actualizar la baseline local para incorporar las huellas digitales de sus propias versiones maliciosas, logrando de esta manera que las verificaciones subsiguientes no detecten anomalía alguna. Este fenómeno, que podría denominarse compromiso del oráculo de confianza, invalida la totalidad de la cadena de verificación y constituye una debilidad estructural del enfoque artesanal.

La tercera limitación, de naturaleza operacional, radica en que los scripts artesanales rara vez se integran con los flujos de trabajo de respuesta a incidentes ni con los canales de comunicación empresarial habituales. La consecuencia directa de esta desconexión es la elevación del Tiempo Medio de Respuesta y el incremento de la probabilidad de que los incidentes reales sean confundidos con falsos positivos rutinarios, perdidos entre correos electrónicos automatizados que los operadores aprenden a filtrar por exceso.

En conjunto, estas tres limitaciones configuran el problema concreto al cual el presente trabajo pretende ofrecer una respuesta tecnológica viable y replicable. La pregunta de investigación orientadora puede formularse entonces de la siguiente manera: ¿es posible diseñar una plataforma de Monitoreo de Integridad de Archivos de código abierto que opere en tiempo real mediante detección reactiva del núcleo del sistema operativo, que proteja la baseline mediante separación arquitectónica y cifrado autenticado, que incorpore decisión

automatizada graduada junto con validación humana formalizada, y que se adecúe a los requerimientos del marco normativo argentino vigente?

1.3 Justificación

La relevancia del trabajo se sustenta en cuatro dimensiones complementarias. La primera dimensión corresponde a la exploración sistemática de la aplicabilidad de plataformas de orquestación de flujos de trabajo al dominio especializado de la seguridad operacional, un área poco sistematizada en la literatura académica hispanohablante contemporánea. La segunda dimensión se vincula con la pertinencia de ofrecer una alternativa tecnológica concreta y de costo accesible para organizaciones sujetas a exigencias crecientes de cumplimiento normativo. La tercera dimensión corresponde al aprovechamiento pedagógico del proyecto dentro del plan de estudios de la Tecnicatura Universitaria en Programación, en tanto articula competencias provenientes de programación en Python, desarrollo web con React y TypeScript, diseño y consulta de bases de datos relacionales, integración de sistemas mediante protocolos estándar y principios de seguridad informática aplicada. La cuarta dimensión corresponde a la contribución al corpus de conocimiento sobre arquitecturas de seguridad específicamente adecuadas al marco normativo argentino, un espacio con escasa producción académica previa.

1.4 Objetivo general

El objetivo general del presente Trabajo Final de carrera consiste en diseñar una plataforma automatizada para el Monitoreo de Integridad de Archivos con arquitectura distribuida, orientada a sistemas Linux, que detecte en tiempo real modificaciones no autorizadas sobre archivos críticos mediante el subsistema fanotify del núcleo, que aplique un motor de decisión local con cuatro niveles de respuesta automatizada, que valide las alteraciones mediante comparación criptográfica contra una baseline cifrada, que incorpore validación humana sobre un flujo formal de eventos con resolución de concurrencia por bloqueo optimista, que notifique los incidentes mediante canales integrados orquestados por n8n con fallbacks automáticos ante indisponibilidad, y que especifique un protocolo experimental reproducible; todo ello en cumplimiento con los requerimientos de la Ley 25.326 y de la normativa argentina aplicable en materia de ciberseguridad.

1.5 Objetivos específicos

Con el propósito de alcanzar el objetivo general anteriormente formulado se proponen los siguientes objetivos específicos. El primero consiste en analizar los fundamentos teóricos de las funciones hash criptográficas resistentes a colisiones, con énfasis en el estándar SHA-256 y su aplicación al campo de la integridad de archivos. El segundo consiste en caracterizar el estado del arte de las soluciones de Monitoreo de Integridad de Archivos disponibles durante el bienio 2024-2026, identificando sus fortalezas y limitaciones.

El tercer objetivo específico consiste en sistematizar el marco normativo argentino aplicable, comprendiendo la Ley 25.326 de Protección de Datos Personales, la Ley 26.388 de Delitos Informáticos, las Resoluciones 47/2018 y 126/2024 de la Agencia de Acceso a la Información Pública, la Resolución 44/2023 de la Secretaría de Innovación Pública, la Decisión Administrativa 641/2021, y el Decreto de Necesidad y Urgencia 941/2025. El cuarto objetivo consiste en formular un modelo de amenazas sobre el propio sistema conforme a la metodología STRIDE, evitando la paradoja habitual de sistemas de protección que no auditan sus propias debilidades.

El quinto objetivo consiste en diseñar una arquitectura distribuida con seis componentes desacoplados comunicados mediante mensajería asíncrona, en la que el agente se despliega como servicio nativo del anfitrión monitoreado mientras que el resto de los componentes se contenedoriza en un servidor central. El sexto objetivo es especificar el protocolo de entrega con garantía exactamente-una-vez para los eventos transportados sobre Valkey Streams. El séptimo objetivo consiste en diseñar el agente de monitoreo con su motor de decisión local, con cola offline acotada, con registro de acciones previas a su ejecución y con cifrado autenticado de la baseline en disco.

El octavo objetivo consiste en diseñar el backend bajo una organización en capas que adopta los patrones de Unit of Work y Repository, con bloqueo optimista sobre la tabla de eventos y con máquina de estados explícita. El noveno objetivo consiste en especificar la arquitectura criptográfica completa del sistema, comprendiendo la autenticación mutua mediante certificados TLS con autoridad certificadora propia, la firma HMAC de comandos y la política de parámetros Argon2id para las contraseñas de los operadores. El décimo objetivo consiste en especificar la política de notificaciones con n8n como orquestador principal y canales alternativos de respaldo automáticos. El undécimo y último objetivo consiste en

formular un protocolo experimental reproducible con grupo de control, con variables, con indicadores operacionalizados y con planillas de captura completas.

1.6 Hipótesis y criterios de aceptación

La hipótesis central del presente trabajo se formula como proposición falsable: la integración de los componentes descriptos permitirá construir una plataforma cuyos indicadores de desempeño, una vez completada la fase experimental, se situarían dentro de los umbrales de aceptación preestablecidos a priori, superando las capacidades operativas típicas de los esquemas tradicionales basados en verificación periódica. La hipótesis nula, correlativa, establece que uno o más indicadores no alcanzan su umbral, o bien que la mejora respecto del enfoque por encuestamiento no resulta estadísticamente significativa tras la ejecución del protocolo experimental.

Los umbrales se fijan a priori con base en valores reportados en la literatura especializada y en los requerimientos operativos típicos de centros de operaciones de seguridad de porte medio. La Tabla 1 sintetiza los indicadores y sus umbrales correspondientes, junto con el fundamento conceptual que respalda cada valor.

Tabla 1. Criterios de aceptación preestablecidos

Indicador	Umbral de aceptación	Fundamento
Latencia de detección (percentil 99)	Menor a 1.000 ms	Escala sub-segundo percibida como tiempo real
Tiempo de notificación (percentil 99)	Menor a 5.000 ms	Compatible con alerta sin acumulación
Cobertura de automatización del triage	Al menos 80 %	Objetivo típico de programas SOAR maduros
Falsos negativos en prueba controlada	Cero	Requisito mínimo de validez del sistema
Recuperación post-falla del agente	Menor a 30 segundos	Resiliencia operativa mínima aceptable

Nota. Elaboración propia. Los umbrales requieren verificación mediante la ejecución del protocolo experimental sobre el laboratorio definitivo, una vez completada la fase de implementación.

1.7 Alcance y limitaciones

El alcance del presente trabajo se circunscribe al diseño detallado y a la especificación del protocolo experimental de un sistema orientado a ambientes Linux con núcleo en su versión 5.1 o superior, requerimiento derivado de las capacidades necesarias del subsistema fanotify adoptado como mecanismo de detección reactiva. La evaluación se prevé sobre un entorno virtualizado de laboratorio, sin contemplar entornos productivos reales ni sistemas de concurrencia industrial. La solución se orienta al monitoreo de directorios críticos del sistema y a paths configurables para aplicaciones.

Quedan excluidos deliberadamente del alcance los siguientes aspectos: el monitoreo de sistemas de archivos distribuidos como NFS o GlusterFS; el monitoreo de contenedores efímeros y de volúmenes persistentes en proveedores de nube pública; la respuesta activa a nivel de red; y la integración productiva con plataformas SIEM comerciales.

Entre las limitaciones metodológicas y técnicas se enumeran las siguientes: la imposibilidad constitucional de fanotify de detectar modificaciones realizadas mediante manipulación directa del dispositivo de bloque subyacente; la dependencia operacional sobre la disponibilidad del backend y del motor de orquestación durante las ventanas no offline; y la ausencia de comparación experimental directa con soluciones comerciales debido a restricciones de acceso al software propietario.

El proyecto se encuentra en fase de documentación y diseño detallado; la implementación del código fuente y la ejecución del protocolo experimental se proyectan para la fase inmediata siguiente del proyecto. En consecuencia, el Capítulo 5 documenta el protocolo experimental y las planillas de captura de datos en lugar de resultados numéricos. Esta decisión, que prioriza la honestidad metodológica sobre la fabricación de datos no verificados, se discute detalladamente en el apartado 5.1 del presente documento y se complementa con el compromiso de publicación de una adenda de resultados, formalizado en el Anexo F.

CAPÍTULO 2. MARCO TEÓRICO

«Assume breach. Verify explicitly. Grant least privilege.»

— Principios de la Arquitectura Zero Trust, NIST SP 800-207 (2020)

El presente capítulo sistematiza los fundamentos conceptuales, técnicos y normativos sobre los cuales se asienta la propuesta desarrollada. La exposición se organiza en trece apartados que siguen una progresión desde los conceptos más abstractos hacia las tecnologías específicas adoptadas y el marco normativo argentino aplicable. Las definiciones técnicas referenciadas a lo largo del texto se amplían en el Glosario provisto en el Capítulo 10, al que se remite al lector no especializado cuando resulta oportuno.

2.1 Seguridad informática y la tríada CIA

La seguridad informática constituye una disciplina cuyo objeto es el diseño, la implementación y la evaluación de mecanismos destinados a preservar las propiedades deseables de los sistemas frente a amenazas de origen intencional o accidental. La evolución de la disciplina ha estado marcada por desplazamientos sucesivos de énfasis: desde la protección física de sistemas centralizados, hacia la protección perimetral mediante cortafuegos, y actualmente hacia paradigmas como la arquitectura de confianza cero, formalizada en la publicación especial SP 800-207 del Instituto Nacional de Estándares y Tecnología de los Estados Unidos (Rose et al., 2020), que asume como principio rector que ningún actor merece confianza a priori, con independencia de su ubicación dentro o fuera del perímetro tradicional.

La tríada conocida por las siglas CIA, por sus términos en inglés, ocupa una posición canónica como marco organizador de los objetivos de protección. Sus tres componentes son la confidencialidad, entendida como garantía de que la información solo resulta accesible a entidades autorizadas; la integridad, propiedad central del presente trabajo, que garantiza la inalterabilidad de la información respecto de un estado de referencia legítimo; y la disponibilidad, que asegura el acceso oportuno de las entidades autorizadas a la información que necesitan.

La integridad admite dos descomposiciones analíticas conceptualmente distintas. La integridad de origen alude a la autenticación del emisor de un mensaje o del autor de un

artefacto, y se resuelve canónicamente mediante firmas digitales y certificados X.509 dentro de infraestructuras de clave pública. La integridad de contenido, por su parte, alude a la inalterabilidad del artefacto respecto de un estado de referencia, y constituye el objeto específico de los sistemas de Monitoreo de Integridad de Archivos analizados en el apartado 2.3.

2.2 Funciones hash criptográficas

Una función hash criptográfica es un procedimiento matemático determinístico que transforma una entrada de longitud arbitraria en una salida de longitud fija denominada resumen o digest. Toda función hash criptográfica debe satisfacer tres propiedades fundamentales. La resistencia a la preimagen asegura que, dado un resumen, resulta computacionalmente inviable reconstruir la entrada que lo generó. La resistencia a la segunda preimagen garantiza que, dado un mensaje, resulta inviable encontrar un segundo mensaje distinto que produzca el mismo resumen. La resistencia a colisiones establece que resulta inviable encontrar un par arbitrario de mensajes distintos cuyos resúmenes coincidan.

El presente trabajo adopta el estándar SHA-256, miembro de la familia SHA-2 estandarizada en la publicación FIPS 180-4 por el Instituto Nacional de Estándares y Tecnología de los Estados Unidos (NIST, 2015). La elección obedece a tres razones convergentes: su amplia disponibilidad en las bibliotecas criptográficas estándar de los lenguajes de programación habituales, incluyendo el módulo hashlib de la biblioteca estándar de Python; el equilibrio adecuado entre velocidad de cómputo y robustez criptográfica; y la ausencia, al momento de la presente redacción, de ataques prácticos conocidos contra sus propiedades fundamentales.

La obsolescencia criptográfica de SHA-1 fue demostrada por Leurent y Peyrin mediante el primer ataque práctico de colisión de prefijo elegido sobre dicha función (Leurent & Peyrin, 2020), motivo por el cual su uso se encuentra proscrito para propósitos de integridad criptográfica en normativas y estándares actualizados. Funciones más recientes como BLAKE3 ofrecen mejoras sustanciales de rendimiento y se consideran como línea de trabajo futuro para la evaluación comparativa.

2.3 Monitoreo de Integridad de Archivos

Un sistema de Monitoreo de Integridad de Archivos se estructura en dos fases funcionales interdependientes. La primera fase es el establecimiento de una baseline, que consiste en calcular las huellas digitales criptográficas de un conjunto de archivos en un estado considerado legítimo y almacenarlas de forma segura. La segunda fase es la verificación continua, que se apoya en recálculos posteriores, realizados periódicamente o como reacción a eventos del sistema operativo, cuya comparación contra la baseline permite detectar alteraciones.

La literatura distingue dos modalidades arquitectónicas principales. La verificación por encuestamiento opera según intervalos programados, lo que introduce ventanas temporales de ceguera entre verificaciones sucesivas. La verificación reactiva basada en eventos se apoya, en cambio, en mecanismos del sistema operativo que notifican en tiempo real las operaciones sobre archivos, con lo cual la detección ocurre con latencia del orden de los milisegundos. El sistema propuesto en el presente trabajo se inscribe en esta segunda modalidad.

2.4 Marco normativo argentino aplicable

La propuesta se enmarca en un entorno normativo argentino que ha experimentado una evolución significativa en los años recientes. El vértice del ordenamiento corresponde al artículo 43 de la Constitución Nacional, que consagra la acción de hábeas data y reconoce la protección de los datos personales como derecho fundamental ligado a la autodeterminación informativa.

La Ley 25.326 de Protección de Datos Personales constituye la columna vertebral del régimen. Su artículo 9.º, que establece la obligación del responsable del tratamiento de adoptar medidas técnicas y organizativas para garantizar la seguridad y confidencialidad de los datos, y en particular de evitar su adulteración, constituye el fundamento legal directo de los sistemas de Monitoreo de Integridad de Archivos en el ordenamiento argentino. La vinculación es explícita: el sistema propuesto implementa una medida técnica orientada a detectar adulteraciones, objetivo que la ley impone como deber del responsable del tratamiento. El texto legal se encuentra vigente con actualizaciones incorporadas hasta marzo de 2026.

La Ley 26.388 de Delitos Informáticos tipifica penalmente el acceso ilegítimo a sistemas, el daño informático, la alteración, destrucción o inutilización de datos, y otras

conductas relevantes en el dominio de aplicación del sistema (Honorable Congreso de la Nación Argentina, 2008). Los registros de auditoría que el sistema conserva con retención ilimitada, cuya especificación se detalla en el Capítulo 4, pueden constituir evidencia digital útil en investigaciones ejercidas bajo este marco.

La Resolución 47/2018 de la Agencia de Acceso a la Información Pública aprobó las medidas de seguridad recomendadas para el tratamiento de datos personales en medios informatizados, organizadas en cinco procesos: recolección, control de acceso, conservación, destrucción y gestión de incidentes (AAIP, 2018). La plataforma propuesta contribuye al proceso de control de acceso mediante trazabilidad completa de las modificaciones en la tabla de auditoría descrita en el Capítulo 4, y al proceso de gestión de incidentes mediante detección temprana y notificación automatizada.

La Resolución 126/2024 de la AAIP actualizó el régimen sancionatorio bajo la Ley 25.326, estableciendo una clasificación tripartita en leves, graves y muy graves (AAIP, 2024). Entre las infracciones leves se incorporó la omisión de informar las medidas de seguridad implementadas, lo que refuerza la relevancia de contar con controles técnicos auditables como los provistos por la plataforma diseñada.

La Resolución 44/2023 de la Secretaría de Innovación Pública aprobó la Segunda Estrategia Nacional de Ciberseguridad, cuyos objetivos incluyen el fortalecimiento de las capacidades de prevención, detección y respuesta, y la protección de las infraestructuras críticas (Secretaría de Innovación Pública, 2023). La Decisión Administrativa 641/2021 de la Jefatura de Gabinete de Ministros aprobó los Requisitos Mínimos de Seguridad de la Información para organismos del Sector Público Nacional (Jefatura de Gabinete de Ministros, 2021), estableciendo un conjunto de controles técnicos que enmarcan el tipo de sistema aquí propuesto.

Finalmente, el Decreto de Necesidad y Urgencia 941/2025, publicado el 2 de enero de 2026, creó el Centro Nacional de Ciberseguridad como organismo descentralizado con rango de autoridad nacional en la materia, en órbita de la Secretaría de Innovación, Ciencia y Tecnología de la Jefatura de Gabinete de Ministros (Poder Ejecutivo Nacional, 2025). La separación formal entre ciberseguridad y ciberinteligencia que introduce el decreto, junto con la posterior designación de autoridades mediante el Decreto 92/2026, consolida la autoridad nacional ante la cual podrían eventualmente reportarse incidentes detectados por sistemas como el aquí propuesto.

2.5 Estado del arte de soluciones FIM

El panorama contemporáneo de las soluciones de Monitoreo de Integridad de Archivos puede caracterizarse mediante cuatro categorías conceptualmente distintas. La primera categoría corresponde a las herramientas de código abierto clásicas, como Tripwire y AIDE, que operan mediante baseline local y encuestamiento periódico. La segunda corresponde a las suites integradas de detección de intrusiones a nivel de anfitrión, como OSSEC y su sucesor Wazuh, que ofrecen integración con otros controles de seguridad pero requieren mayor complejidad de despliegue. La tercera corresponde a las soluciones comerciales empresariales, como Tripwire Enterprise o CrowdStrike Falcon, con funcionalidad madura pero fuera del alcance económico típico de organizaciones medianas. La cuarta categoría, de aparición reciente, no constituye propiamente un FIM pero se menciona frecuentemente en contextos adyacentes: se trata de las herramientas de seguridad en tiempo de ejecución basadas en eBPF, como Falco y Tetragon, que operan sobre el flujo de llamadas al sistema del núcleo y cuya comparación con la presente propuesta se desarrolla en el apartado 6.7.

La propuesta del presente trabajo toma la accesibilidad económica y filosófica del software libre clásico, toma la filosofía de integración modular de las suites de detección de intrusiones, y aspira, mediante la orquestación con n8n y la integración con canales operativos, a una parte de la sofisticación funcional propia de las soluciones comerciales.

2.6 Subsistemas del núcleo Linux para detección de eventos de archivo

El núcleo Linux provee distintos subsistemas para notificar a los procesos de usuario sobre operaciones realizadas sobre archivos. El más antiguo y conocido es inotify, incorporado en 2005, que opera mediante un descriptor de archivo sobre el cual el proceso de usuario recibe eventos asíncronos (Kernel Linux, 2025). El subsistema inotify presenta, sin embargo, limitaciones conocidas: no opera de manera recursiva de forma nativa, lo que obliga a registrar observadores por cada subdirectorio; el parámetro del kernel `fs.inotify.max_user_watches` limita el número de observadores simultáneos por usuario; el parámetro `fs.inotify.max_queued_events` limita el tamaño de la cola interna, cuyo desbordamiento emite el evento `IN_Q_OVERFLOW` con pérdida silenciosa de eventos posteriores; no opera sobre sistemas de archivos remotos montados mediante NFS o CIFS; y

no detecta cambios realizados mediante manipulación directa del dispositivo de bloque subyacente.

El subsistema fanotify, incorporado en 2010 y sustancialmente mejorado en la versión 5.1 del núcleo, se diseñó específicamente para sistemas de seguridad. A diferencia de inotify, fanotify puede operar en modo notificación pura o en modo con contenido previo, en el cual el núcleo suspende la ejecución de la llamada al sistema hasta que el proceso de usuario responde permitiendo o rechazando la operación. Esta última modalidad habilita la prevención efectiva de escrituras no autorizadas, a diferencia del paradigma reactivo tradicional que solo detecta la alteración una vez consumada. Adicionalmente, fanotify provee información del proceso causante del evento dentro del propio mensaje, incluyendo el identificador de proceso, el identificador de usuario y la ruta del ejecutable, con lo cual desaparece la necesidad de correlacionar con herramientas externas como auditd. El subsistema soporta el monitoreo de sistemas de archivos completos mediante una única llamada con la marca `FAN_MARK_FILESYSTEM`, eliminando así la condición de carrera propia de inotify al registrar observadores sobre subdirectorios creados dinámicamente.

La contrapartida de fanotify es la necesidad de la capacidad de núcleo `CAP_SYS_ADMIN`, dado el carácter privilegiado de sus operaciones. En entornos contenedorizados esta exigencia compromete el aislamiento propio del contenedor, lo cual motiva la decisión arquitectónica de desplegar el agente como servicio nativo del anfitrión monitoreado, detallada en el Capítulo 4. El presente trabajo adopta fanotify como subsistema primario mediante el wrapper de Python denominado `pyfanotify`, en su versión 0.3.0 publicada en julio de 2024 y mantenida activamente, con licencia MIT, con implementación nativa en lenguaje C y documentación vigente. Se descartó el wrapper alternativo `python-fanotify` de Google por encontrarse su repositorio oficialmente archivado.

2.7 Paradigma SOAR y orquestación con n8n

El paradigma Security Orchestration, Automation and Response, conocido por el acrónimo SOAR, emergió como respuesta sistemática a la saturación del personal humano ante el volumen de alertas de seguridad. Una plataforma SOAR conjuga tres dimensiones complementarias: la orquestación de herramientas heterogéneas, la automatización de tareas repetitivas y la gestión estructurada de la respuesta mediante flujos formalizados denominados playbooks. La publicación especial SP 800-61 en su revisión 2, elaborada por el NIST,

estableció un marco metodológico para el manejo de incidentes que continúa siendo la referencia conceptual de los playbooks modernos (Cichonski et al., 2012).

La plataforma n8n es un motor de automatización de flujos de trabajo publicado bajo licencia de uso sostenible, categoría fair-code no equivalente a software libre puro, cuya arquitectura se organiza en torno al concepto de flujo como grafo dirigido de nodos que representan servicios e integraciones. En el diseño del presente trabajo, n8n se emplea de manera acotada como enrutador de notificaciones externas: recibe webhooks desde el backend y los reencamina hacia canales heterogéneos como correo electrónico o plataformas de mensajería corporativa. Esta delimitación responde a una decisión arquitectónica consciente: n8n no asume el rol de coordinador central del playbook sino el de adaptador flexible de notificaciones, lo que permite aprovechar su ecosistema de integraciones sin introducir una dependencia crítica en el camino principal de procesamiento. La política de fallbacks automáticos, detallada en el apartado 4.7, garantiza además que la indisponibilidad de n8n no comprometa la continuidad del servicio de alertado.

2.8 Stack backend: FastAPI, SQLAlchemy y PostgreSQL

FastAPI es un framework web moderno para el lenguaje Python cuya propuesta de valor se centra en tres aspectos: alto rendimiento mediante operación asíncrona sobre el estándar ASGI, generación automática de documentación conforme a la especificación OpenAPI, y validación automática de datos mediante el paquete Pydantic. SQLAlchemy, desarrollado por el mismo autor, unifica en un modelo único las capacidades de un ORM construido sobre SQLAlchemy y las de un sistema de validación basado en Pydantic. La unificación simplifica el diseño al evitar la duplicación habitual entre modelos de persistencia y modelos de API.

PostgreSQL en su versión 18 asume el rol de repositorio centralizado del estado autoritativo del sistema: metadatos de la baseline, eventos históricos, reglas de monitoreo, alertas, acciones administrativas, credenciales de operadores, tabla de auditoría con retención ilimitada, y registro de certificados revocados. El controlador adoptado es psycopg en su versión 3, controlador oficial con soporte tanto sincrónico como asíncrono. La ubicación del repositorio PostgreSQL en un servidor central, físicamente remoto respecto de los anfitriones monitoreados, desacopla el almacenamiento del estado autoritativo del propio sistema

potencialmente vulnerable, contribuyendo a mitigar el compromiso del oráculo de confianza descrito en el Capítulo 1.

2.9 Mensajería con Valkey Streams

Valkey es un almacén de estructura de datos en memoria de código abierto, surgido en 2024 como derivación del proyecto Redis tras el cambio de licenciamiento de este último. Opera bajo la gobernanza de la Linux Foundation y mantiene compatibilidad binaria con los clientes existentes del ecosistema Redis (Valkey Contributors, 2026). La funcionalidad central que aporta al diseño son los streams, una estructura de datos persistente y ordenada temporalmente que provee semántica de entrega al-menos-una-vez.

El sistema emplea tres streams diferenciados. El stream denominado events transporta eventos desde los agentes hacia el backend. El stream commands transporta comandos desde el backend hacia los agentes, incluyendo actualizaciones de baseline, sincronización de reglas, solicitudes de restauración y confirmaciones de recepción. El stream agent_heartbeat transporta latidos periódicos que permiten al backend detectar agentes caídos o en estado degradado.

Conviene explicitar un aspecto conceptual relevante del diseño. Valkey cumple en la arquitectura el rol de medio de transporte, no de almacenamiento autoritativo de la verdad. La durabilidad extremo a extremo se garantiza mediante la combinación de la cola local en disco del agente, que retiene los eventos hasta recibir confirmación explícita de persistencia, y la base de datos PostgreSQL, que almacena el estado una vez persistido. Cualquier falla intermedia de Valkey se recupera mediante reenvío desde el agente, y cualquier duplicado eventual se descarta por restricción de unicidad sobre el identificador UUID del evento.

2.10 Primitivas criptográficas adoptadas

El presente trabajo adopta un conjunto de primitivas criptográficas orientadas a distintos propósitos. Para el hashing de contraseñas de operadores administrativos se emplea Argon2id, función de hashing con consumo intensivo de memoria ganadora de la Password Hashing Competition en 2015 y estandarizada en el documento RFC 9106 (Biryukov et al., 2021). Los parámetros operativos de referencia se exponen en el Capítulo 4, junto con el esquema de parametrización por entorno adoptado.

Para el cifrado en reposo del contenido de la baseline almacenada en el sistema de archivos local del agente se emplea AES-256-GCM, cifrado simétrico autenticado que provee confidencialidad e integridad simultáneamente en una única operación. La clave simétrica correspondiente se deriva mediante la función HKDF-SHA256 estandarizada en el documento RFC 5869 (Krawczyk & Eronen, 2010), tomando como material de entrada un secreto maestro único por agente, el identificador del agente como sal y una cadena de propósito como información de contexto. El secreto maestro se entrega al agente durante el proceso de arranque inicial y se persiste localmente con permisos de archivo que restringen el acceso al usuario propietario exclusivamente. Esta decisión resulta arquitectónicamente relevante: el compromiso del sistema de archivos local, sin la clave maestra, no permite leer ni manipular la baseline de forma verificable.

Para dos propósitos adicionales se emplea HMAC-SHA256. Primero, durante el arranque inicial del agente, para firmar la solicitud de firma de certificado enviada al backend con un secreto de arranque compartido fuera de banda. Segundo, para firmar cada comando publicado por el backend en el stream commands, lo que permite al agente rechazar comandos espurios incluso ante compromiso hipotético del propio motor Valkey. Esto configura una defensa en profundidad: la autenticación mutua mediante certificados TLS autentica el canal, y la firma HMAC autentica el mensaje individual.

La autenticación entre agentes y backend se implementa mediante autenticación mutua TLS sobre la versión 1.3 del protocolo, estandarizada en el documento RFC 8446 (Rescorla, 2018), con autoridad certificadora propia gestionada por el backend, cuyos detalles operativos se describen en el Capítulo 4.

2.11 Zero Trust y raíz de confianza

La arquitectura de confianza cero rechaza la noción tradicional de perímetro confiable y exige verificación explícita de cada interacción. Sus tres principios fundamentales, a saber, verificación explícita, privilegio mínimo y asunción de compromiso, orientan el diseño del presente trabajo (Rose et al., 2020). La plataforma aborda primariamente los pilares de dispositivos, mediante autenticación criptográfica del agente por certificados TLS mutuos, y de datos, mediante verificación continua de integridad, baseline cifrada en reposo y comandos firmados HMAC.

Resulta necesario explicitar una limitación inherente al diseño. El agente opera en espacio de usuario y depende de la integridad del núcleo del sistema operativo. Un adversario con capacidad de ejecutar código en espacio de núcleo puede eludir fanotify, modificar archivos directamente mediante operaciones a nivel de dispositivo de bloque, o interferir con el proceso del agente. La raíz de confianza efectiva se establece entonces en el núcleo Linux no comprometido. Extender la raíz de confianza a un nivel inferior requiere mecanismos complementarios como Integrity Measurement Architecture, dm-verity o Secure Boot, que constituyen líneas de trabajo futuro documentadas en el Capítulo 8.

2.12 Modelo de amenazas STRIDE

El modelo STRIDE proporciona un marco sistemático para la identificación de amenazas mediante seis categorías: suplantación, manipulación, repudio, divulgación de información, denegación de servicio y elevación de privilegios. Su aplicación al propio sistema FIM resulta metodológicamente indispensable para evitar la paradoja de un sistema de protección que no audita sus propias debilidades. La Tabla 2 sintetiza el análisis realizado junto con las mitigaciones de diseño descriptas en el Capítulo 4.

Tabla 2. Modelo STRIDE aplicado al sistema FIM

Amenaza	Aplicación al sistema	Mitigación por diseño
Suplantación	Un agente falso emite eventos; un atacante emite comandos al agente.	Autenticación mutua TLS con CA propia; firma HMAC sobre comandos; bootstrap con CSR firmado.
Manipulación	Manipulación de baseline en disco; del binario del agente; de eventos en tránsito.	Cifrado AES-GCM de baseline con clave derivada HKDF; versionado de esquema de mensajes; protocolo ACK bidireccional.
Repudio	Operador niega haber aprobado un cambio; aprobaciones concurrentes.	Tabla de auditoría separada con retención ilimitada; bloqueo optimista que rechaza la segunda aprobación; campos resolved_at y resolved_by firmados en el evento.
Divulgación	Exposición de baseline; exposición de logs con información sensible.	Cifrado en reposo de baseline; sanitización de logs; cookies httpOnly; política de seguridad de contenido CSP.
Denegación de servicio	Inundación de eventos, webhooks o intentos de inicio de sesión.	Limitación de tasa multicapa; cola local acotada con política drop-oldest; reintentos exponenciales con cola de mensajes fallidos.

Amenaza	Aplicación al sistema	Mitigación por diseño
Elevación de privilegios	Compromiso del backend; bypass del flujo de aprobación.	Segmentación mediante redes Docker; cambio forzado de contraseña de seed; máquina de estados explícita en el backend.
Replay (amenaza complementaria)	Reinyección de eventos o comandos antiguos.	Timestamps dobles con tolerancia menor a 5 minutos; lista negra de identificadores jti en Valkey; versión de ruleset monotónicamente creciente.

Nota. Elaboración propia. La amenaza residual principal corresponde a rootkits del núcleo, cuya mitigación excede el alcance de un FIM en espacio de usuario y se discute en el Capítulo 8.

2.13 Patrones de diseño de software aplicados

El backend del sistema adopta una organización en capas inspirada en los principios de Arquitectura Limpia. En esta organización, la capa de presentación responde al protocolo HTTP mediante los routers de FastAPI; la capa de aplicación contiene los casos de uso que implementan las operaciones de negocio; la capa de dominio modela las entidades y las reglas de negocio puras, sin conocimiento de tecnologías concretas; y la capa de infraestructura provee las implementaciones técnicas de acceso a base de datos, mensajería y servicios externos.

Dos patrones específicos resultan centrales para la disciplina de acceso a datos. El patrón Repository encapsula en clases dedicadas toda la lógica de acceso a la base de datos para cada entidad del dominio, de modo que los casos de uso pueden solicitar operaciones sobre eventos, reglas o auditoría sin acoplarse al detalle SQL. El patrón Unit of Work agrupa múltiples operaciones de repositorios bajo una misma transacción de base de datos, garantizando atomicidad mediante confirmación única al completar el caso de uso o reversión completa ante cualquier error intermedio. La adopción conjunta de ambos patrones elimina por diseño la categoría de errores en la cual un desarrollador olvida una reversión explícita tras una excepción, y facilita la verificación mediante pruebas unitarias con repositorios simulados en memoria.

CAPÍTULO 3. MARCO METODOLÓGICO

«El único experimento que merece la pena realizar es aquel que pueda demostrar que estás equivocado.»

— Adaptado y traducido de Wohlin et al., Experimentation in Software Engineering (2012)

En el presente capítulo se describe la estrategia metodológica adoptada para el diseño y la futura evaluación empírica de la plataforma. Se explicitan el tipo de investigación y el enfoque metodológico en subsecciones diferenciadas, se justifica el diseño experimental con grupo de control, se definen las variables e indicadores que operacionalizan la hipótesis, se enumeran las técnicas e instrumentos de recolección, se describe el conjunto de herramientas utilizadas como subsección dedicada, y se detalla el protocolo experimental reproducible conforme a los estándares metodológicos de la investigación aplicada en ingeniería informática.

3.1 Tipo de investigación

La investigación se clasifica, atendiendo a su finalidad, como investigación aplicada: su propósito es resolver un problema concreto del contexto organizacional mediante la construcción y la evaluación de un artefacto tecnológico específico. Se distingue así de la investigación básica, cuyo objetivo es la ampliación del conocimiento teórico sin requerimiento de aplicación inmediata.

Atendiendo al nivel de profundidad, el trabajo se inscribe simultáneamente en los niveles descriptivo y explicativo. El nivel descriptivo se manifiesta en la caracterización del fenómeno del compromiso de integridad de archivos y en la descripción detallada del artefacto propuesto. El nivel explicativo se manifiesta en el análisis de las relaciones causales entre las decisiones de diseño arquitectónico y los indicadores de desempeño esperados, relación que la hipótesis formaliza para su futura verificación empírica.

Atendiendo a la dimensión temporal, se trata de un estudio de corte transversal, con mediciones realizadas durante un período acotado de pruebas controladas sobre un laboratorio definitivo, sin seguimiento longitudinal posterior ni evaluación de deriva del comportamiento a lo largo del tiempo.

3.2 Enfoque metodológico

El enfoque metodológico es predominantemente cuantitativo, en tanto la hipótesis se operacionaliza mediante indicadores numéricos medibles, y las decisiones sobre su confirmación o refutación se toman mediante comparación contra umbrales preestablecidos. La operacionalización concreta de la hipótesis se materializa en las variables e indicadores desarrollados en el apartado 3.4.

El enfoque incorpora, no obstante, elementos cualitativos relevantes en la fase de diseño arquitectónico. Las decisiones sobre organización en capas, patrones de diseño aplicados, mantenibilidad del código, legibilidad de los modelos de datos, facilidad de extensión frente a requerimientos futuros y claridad conceptual de la descomposición modular no se prestan a cuantificación directa. Se justifican, en consecuencia, mediante argumentación técnica apoyada en la literatura especializada y en las decisiones de auditoría interna documentadas por el equipo de diseño.

La coexistencia de ambos enfoques configura un diseño metodológico mixto con predominio cuantitativo, apropiado para trabajos de ingeniería en los cuales el artefacto tecnológico se diseña cualitativamente y se evalúa cuantitativamente, tal como Wohlin y colaboradores caracterizan a la investigación experimental en ingeniería de software (Wohlin et al., 2012).

3.3 Diseño experimental con grupo de control

El presente estudio adopta un diseño experimental con grupo de control para habilitar conclusiones comparativas en lugar de afirmaciones aisladas sobre el desempeño del artefacto. El grupo experimental corresponde a la plataforma propuesta operando en modo reactivo sobre fanotify. El grupo de control corresponde a un script tradicional de verificación periódica, implementado como tarea cron que ejecuta la herramienta sha256sum cada quince minutos sobre los mismos directorios, que almacena la baseline en un archivo local y que compara los resultados contra la verificación anterior mediante la utilidad diff.

El script de control replica deliberadamente las prácticas artesanales descriptas en el planteamiento del problema, incluyendo sus tres limitaciones estructurales: ventana de ceguera por encuestamiento, baseline local vulnerable, y ausencia de notificación automatizada. Ambos sistemas se evalúan sobre el mismo entorno de laboratorio, con la

misma carga de trabajo generada por el mismo instrumento, durante ventanas de medición equivalentes. Esta configuración permite aislar el efecto de las decisiones arquitectónicas del presente trabajo sobre indicadores comparables.

3.4 Variables e indicadores

La operacionalización de la hipótesis se materializa en cinco variables principales con sus indicadores correspondientes. La Tabla 3 presenta las variables operacionales, sus unidades de medida y los indicadores con sus umbrales de aceptación preestablecidos.

Tabla 3. Variables e indicadores experimentales

Variable	Definición operacional	Unidad	Indicador
Latencia de detección	Intervalo entre modificación del archivo y recepción del evento en el backend.	ms	Percentil 99 < 1.000 ms
Tiempo de notificación	Intervalo entre recepción del evento y emisión de la notificación final.	ms	Percentil 99 < 5.000 ms
Cobertura de triage	Porcentaje de tareas del marco NIST SP 800-61r2 automatizadas por el sistema.	%	$\geq 80 \%$
Resiliencia offline	Porcentaje de eventos preservados tras desconexión del backend.	%	100 %
Latencia por polling (control)	Intervalo entre modificación y detección por script cron cada 15 minutos.	ms	Observacional comparativo

Nota. Elaboración propia. Los umbrales se fijaron a priori con base en valores reportados en Cichonski et al. (2012) y en requerimientos operativos típicos de centros de operaciones de seguridad.

3.5 Herramientas utilizadas

El presente apartado compendia, por claridad expositiva y en concordancia con la plantilla CONEAU, el conjunto de herramientas empleadas tanto durante la fase de recolección como durante la fase de procesamiento posterior de los datos experimentales. Las

herramientas se agrupan en tres categorías funcionales según su propósito dentro del protocolo.

La primera categoría comprende las herramientas de generación de carga experimental. El instrumento principal es un generador de carga de archivos desarrollado específicamente para el presente trabajo en lenguaje Python, que crea, modifica y elimina archivos según los parámetros configurables del protocolo (tamaño, distribución temporal, proporción de operaciones). Complementariamente, se emplea la utilidad `tcpdump` del sistema para capturar tráfico de red durante las pruebas, como instrumento de verificación independiente de los timestamps consignados por el propio sistema. La utilidad `strace` proporciona trazas de llamadas al sistema cuando se requiere correlación fina entre eventos `fanotify` y operaciones del agente.

La segunda categoría comprende las herramientas de recolección y registro. El sistema emplea `structlog`, biblioteca Python para logging estructurado en formato JSON, en todos los componentes de software. Las consultas SQL ejecutadas directamente sobre PostgreSQL proveen extracción de métricas agregadas desde las tablas de eventos, alertas y auditoría. El conjunto de scripts de shell automatiza la ejecución secuencial de las baterías del protocolo y la recolección de archivos de traza resultantes.

La tercera categoría comprende las herramientas de procesamiento estadístico. La biblioteca `pandas` de Python provee las capacidades de tratamiento de los datos capturados, particularmente para el cálculo de percentiles, medias, desviaciones y agregaciones cruzadas. La biblioteca `matplotlib` se reserva para la generación de visualizaciones que acompañarán la adenda de resultados prevista una vez completada la ejecución experimental. Las hojas de cálculo en formato abierto ODS provistas en el Capítulo 5 sirven como planillas de captura tabular estructurada, cuya completitud constituye evidencia documentada de la ejecución del protocolo.

3.6 Técnicas de recolección

La recolección de datos se articula en cuatro técnicas complementarias que, tomadas en conjunto, proveen redundancia metodológica y habilitan la verificación cruzada de mediciones críticas.

La primera técnica es la medición directa mediante instrumentación programática del propio sistema: marcas temporales con precisión de nanosegundos se obtienen en los puntos críticos del flujo, incluyendo detección por fanotify, publicación en Valkey con el campo `detected_at`, consumo por el backend con el campo `received_at`, cálculo de hash, consulta a baseline, actualización de estado y emisión de notificación. El diferencial entre `received_at` y `detected_at` provee la latencia extremo a extremo correspondiente a la variable principal del estudio.

La segunda técnica es el análisis de logs estructurados en formato JSON emitidos por todos los componentes del sistema. La tercera técnica es la inspección manual de las notificaciones emitidas hacia los canales configurados, con el propósito de verificar contenido, completitud y oportunidad temporal. La cuarta técnica es la evaluación de cobertura contra la checklist derivada del marco metodológico NIST SP 800-61r2, realizada por los autores del trabajo mediante análisis cualitativo estructurado con criterios objetivados.

3.7 Protocolo experimental

El protocolo se estructura en seis baterías secuenciales ejecutables sobre un laboratorio virtualizado con características estandarizadas. El anfitrión físico de referencia provee un procesador Intel Core i7 de decimotercera generación, 32 gibibytes de memoria RAM, almacenamiento NVMe de 1 terabyte, y sistema operativo Ubuntu Server 24.04 LTS. Sobre este anfitrión se ejecuta el servidor central mediante Docker Compose con los componentes backend, PostgreSQL, Valkey, n8n y frontend, junto con un contenedor separado que aloja a un anfitrión monitoreado simulado donde el agente corre como servicio nativo. El anfitrión físico aloja adicionalmente la tarea cron del grupo de control, aislada en su propio directorio de baseline.

La primera batería corresponde al establecimiento de la baseline inicial mediante el proceso de escaneo del agente sobre los directorios monitoreados, con verificación de consistencia mediante comparación contra sha256sum. La segunda batería ejecuta la validación funcional del backlog: se validan las treinta y una historias de usuario formalmente documentadas, conformadas por veinticuatro originales más siete incorporadas tras la auditoría interna del proyecto.

La tercera batería mide la latencia de detección mediante quinientas modificaciones programáticas con distribución temporal uniforme sobre treinta minutos, ejecutada en paralelo

sobre el grupo experimental y el grupo de control. La cuarta batería mide el tiempo de ejecución del proceso de notificación mediante mil invocaciones secuenciales bajo tres niveles de concurrencia: secuencial, cincuenta concurrentes y cien concurrentes.

La quinta batería evalúa la resiliencia frente a la desconexión del backend durante cinco minutos mientras se generan eventos a una tasa de diez por segundo, verificando que la cola local del agente preserve los eventos en orden y los entregue íntegramente tras la reconexión conforme al protocolo que procesa primero los comandos pendientes y después los eventos encolados. La sexta batería evalúa la cobertura de automatización del triage mediante análisis comparativo contra la checklist derivada del NIST SP 800-61r2. Las planillas de captura correspondientes a las seis baterías se presentan en el Capítulo 5.

CAPÍTULO 4. DESARROLLO DE LA INVESTIGACIÓN

«Semanas de programación pueden ahorrarte horas de planificación.»

— Aforismo anónimo de la ingeniería de software

El presente capítulo documenta el proceso de diseño del sistema mediante la descripción funcional de cada componente y de sus interacciones. La exposición privilegia los flujos de procesamiento y las decisiones arquitectónicas por sobre el detalle de implementación, coherente con el carácter académico del trabajo. Las decisiones técnicas que se describen a continuación incorporan los resultados de una auditoría interna de consistencia, seguridad y resiliencia documentada por el equipo de diseño. Los extractos ilustrativos de esquemas y configuraciones se incluyen en los anexos del Capítulo 12.

Resulta oportuno explicitar, antes de ingresar al contenido sustantivo, una observación sobre la interpretación del presente capítulo respecto del modelo canónico CONEAU. El modelo de referencia ejemplifica el Capítulo 4 mediante elementos vinculados a la descripción del entorno de prueba y a las pruebas ejecutadas sobre un sistema preexistente. Dado que el presente trabajo corresponde a una investigación aplicada orientada a la construcción de un artefacto tecnológico, y no a una auditoría empírica sobre un sistema ajeno, el presente capítulo se reinterpreta como descripción detallada del artefacto diseñado, mientras que la descripción del entorno experimental sobre el cual operará se proporciona en el apartado 3.7 como parte del Marco Metodológico.

4.1 Arquitectura general y distribución física

El sistema se concibe bajo un patrón arquitectónico de microservicios ligeramente acoplados, en el cual cada componente asume una responsabilidad técnica claramente delimitada y se comunica con los restantes exclusivamente a través de interfaces estandarizadas. Esta decisión responde a cuatro objetivos convergentes: la mantenibilidad mediante la evolución independiente de cada componente, la sustitución tecnológica habilitada por el bajo acoplamiento entre piezas, la posibilidad de un despliegue físicamente distribuido, y la aplicación de controles de seguridad específicos a cada interfaz como materialización concreta del principio de defensa en profundidad.

La Figura 1 presenta la arquitectura general del sistema, identificando los seis componentes, los tres canales principales de comunicación y la separación física entre el anfitrión monitoreado y el servidor central.

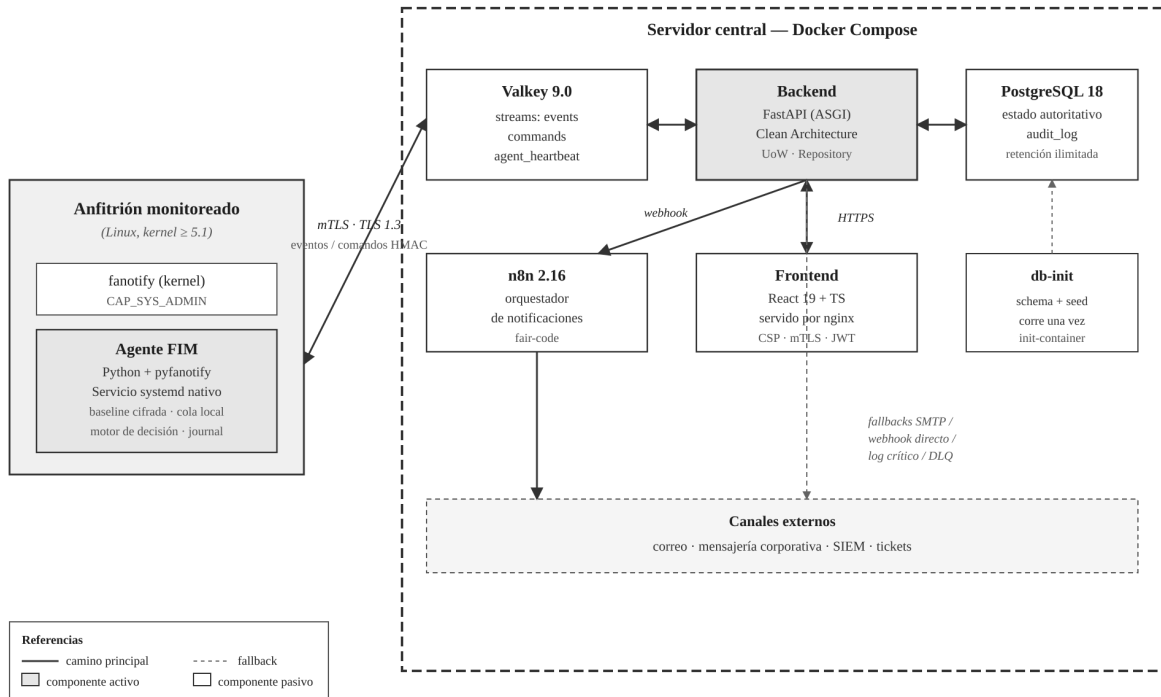


Figura 1. Arquitectura general del sistema y distribución física

Nota. Los agentes operan como servicios nativos sobre los anfitriones monitoreados. El servidor central contenedoriza el backend, PostgreSQL, Valkey, n8n y el frontend. La comunicación entre agente y servidor se realiza sobre TLS 1.3 con autenticación mutua.

La distribución física del sistema opera sobre dos ámbitos físicos diferenciados. El primero es el servidor central, donde se contenedoriza mediante Docker Compose el conjunto de componentes del lado servidor: el backend FastAPI, la base de datos PostgreSQL, el motor de mensajería Valkey, el orquestador de notificaciones n8n, y la aplicación frontend servida por nginx. El segundo son los anfitriones monitoreados, en cada uno de los cuales el agente se despliega como servicio nativo del sistema operativo gestionado por systemd, sin contenedorización. Esta separación obedece a razones técnicas concretas descriptas en el apartado 4.10: el agente requiere visibilidad directa del sistema de archivos del anfitrión, capacidades específicas del núcleo como CAP_SYS_ADMIN para fanotify, y un ciclo de vida acoplado al del anfitrión antes que a un orquestador de contenedores.

El backend se diseña como instancia única por decisión explícita: no se contempla despliegue con alta disponibilidad multi-réplica en el producto mínimo viable. Las asunciones del bloqueo optimista sobre la tabla de eventos, de la limitación de tasa en proceso y del consumer group único de Valkey se sustentan en este supuesto. La Tabla 4 resume el stack tecnológico adoptado con las versiones validadas a la fecha de elaboración del presente documento.

Tabla 4. *Stack tecnológico por componente*

Componente	Tecnología	Versión	Ámbito
Agente de monitoreo	Python + pyfanotify	3.12 + 0.3.0	Nativo en cada anfitrión
Backend API	FastAPI	0.136.0	Servidor central (Docker)
ORM	SQLModel	última estable	Servidor central (Docker)
Controlador DB	psycopg	v3	Servidor central (Docker)
Base de datos	PostgreSQL	18	Servidor central (Docker)
Mensajería	Valkey	9.0	Servidor central (Docker)
Notificaciones	n8n	2.16.1	Servidor central (Docker)
Autenticación usuarios	python-jose + JWT	última estable	Servidor central (Docker)
Hash contraseñas	argon2-cffi	última estable	Servidor central (Docker)
Autenticación agente	mTLS con CA propia	TLS 1.3	Canal agente-backend
Frontend	React + TypeScript	React 19	Servidor central (Docker)
Estilos	Tailwind CSS	4.2	Servidor central (Docker)
Logging	structlog	última estable	Transversal

Componente	Tecnología	Versión	Ámbito
Orquestación	Docker + Compose	última estable	Servidor central
Gestor de servicios del agente	systemd	según distribución	Cada anfitrión

Nota. Las versiones fueron validadas a la fecha del documento de arquitectura del proyecto. La selección se justificó por la madurez, la adopción industrial y la calidad del mantenimiento activo de cada proyecto.

4.2 Esquema de persistencia

El esquema de persistencia en PostgreSQL se diseñó atendiendo a ocho requisitos funcionales derivados del análisis del dominio y de la auditoría de seguridad. El primer requisito corresponde al almacenamiento consultable de las huellas digitales de referencia asociadas a cada archivo monitoreado. Este almacenamiento soporta dos estados posibles: el estado `present` cuando el archivo debe existir con un hash determinado, y el estado `absent` cuando corresponde registrar que un path determinado no debe contener archivo. El segundo requisito es la trazabilidad temporal completa del historial de cambios detectados y sus resoluciones humanas o automáticas. El tercero es el registro de eventos de auditoría de acciones administrativas. El cuarto es la representación de cadenas de eventos encadenados mediante el campo `parent_event_id`, que permite modelar la situación en que un archivo cambia varias veces antes de su resolución humana. El quinto es la gestión de reglas de monitoreo con patrones de ruta y acciones asociadas.

Los tres requisitos restantes responden a decisiones de hardening documentadas por la auditoría interna. El sexto requisito es el soporte para bloqueo optimista sobre la tabla de eventos, mediante una columna `version` entera que se incrementa atómicamente en cada transición de estado. Ante un intento de actualización con versión desactualizada, el sistema retorna el código de estado HTTP 409 que el frontend traduce en un mensaje de refresco. El séptimo requisito es la persistencia de una tabla de auditoría separada de los logs estructurados, con retención ilimitada no sujeta a rotación. Esta decisión se alinea directamente con las exigencias de trazabilidad de la Resolución 47/2018 de la Agencia de Acceso a la Información Pública, en cuyo proceso de control de acceso resulta indispensable disponer de un registro verificable de las acciones administrativas. El octavo requisito es la persistencia de un conjunto de tablas auxiliares que soportan controles específicos: eventos rechazados por desincronización horaria, notificaciones fallidas para su reintento manual, certificados revocados, y un contador de versión de ruleset monotónicamente creciente.

La Figura 2 presenta el modelo entidad-relación simplificado de las tablas principales del esquema, junto con sus claves foráneas y cardinalidades.

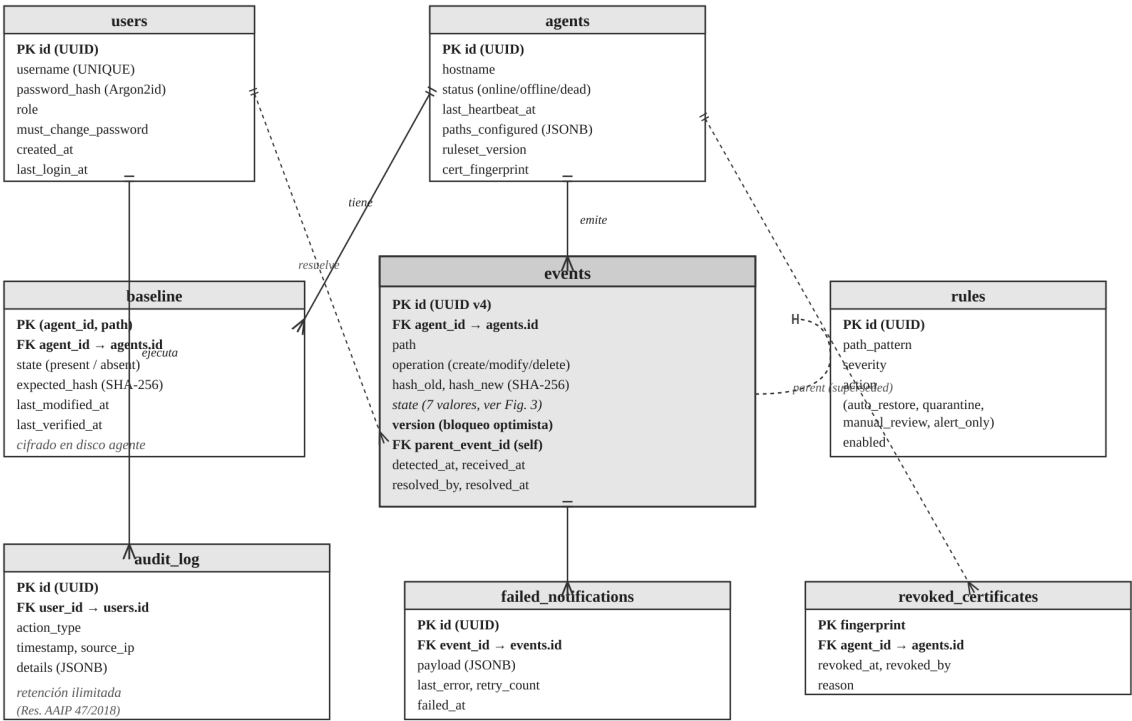


Figura 2. Modelo entidad-relación de las tablas principales de PostgreSQL

Nota. La tabla events ocupa el lugar central del modelo: mantiene referencias hacia agents (agente emisor), users (operador que resuelve) y admite auto-referencia mediante parent_event_id para representar la cadena superseded. La tabla audit_log, con retención ilimitada, materializa la alineación con la Resolución AAIP 47/2018.

Tabla 5. Tablas principales del esquema de persistencia

Tabla	Propósito
users	Credenciales de operadores con flag de cambio forzado de contraseña en el primer inicio de sesión.
agents	Registro de agentes conectados, estado actual, paths configurados y último latido recibido.
baseline	Estado autoritativo por path monitoreado; admite estado present o absent.
events	Núcleo operativo con 7 estados posibles, bloqueo optimista y timestamps dobles.

Tabla	Propósito
rules	Reglas de monitoreo con patrones, severidad y acción configurada.
alerts y actions	Registro de alertas emitidas y acciones administrativas ejecutadas sobre el sistema.
audit_log	Auditoría separada con retención ilimitada alineada con Res. AAIP 47/2018.
failed_notifications	Cola de mensajes fallidos (DLQ) de webhooks n8n para reintento o descarte.
rejected_events_audit	Eventos descartados por desincronización horaria o esquema inválido.
revoked_certificates	Lista de revocación de certificados TLS de agentes.

Nota. La separación entre la tabla de auditoría y los logs estructurados obedece a una decisión deliberada: los logs rotan a 30 días, mientras que la auditoría se conserva indefinidamente.

4.3 Flujo del agente de monitoreo

El agente constituye el componente más estrechamente acoplado al sistema anfitrión y se organiza funcionalmente en seis módulos internos. Un observador configura la escucha de fanotify sobre las rutas configuradas. Un calculador de huellas digitales computa el hash SHA-256 de cada archivo modificado. Un motor de decisión evalúa las reglas cacheadas localmente para determinar la acción a ejecutar. Un subsistema de almacenamiento local gestiona la baseline cifrada, la cola offline, el directorio de cuarentena, el registro de acciones previas y los certificados. Un comunicador publica eventos, consume comandos y emite latidos hacia el servidor central. Un recuperador rehidrata el estado al arranque para garantizar continuidad operativa tras reinicios del anfitrión o del propio servicio.

La Figura 3 presenta el flujo operativo del agente organizado en tres fases temporales bien diferenciadas. La fase de arranque inicializa el estado del agente y prepara la observación. La fase de operación procesa eventos de fanotify conforme a las reglas cacheadas. La fase de sincronización, concurrente con la anterior, gestiona latidos, confirmaciones y rotación de certificados.

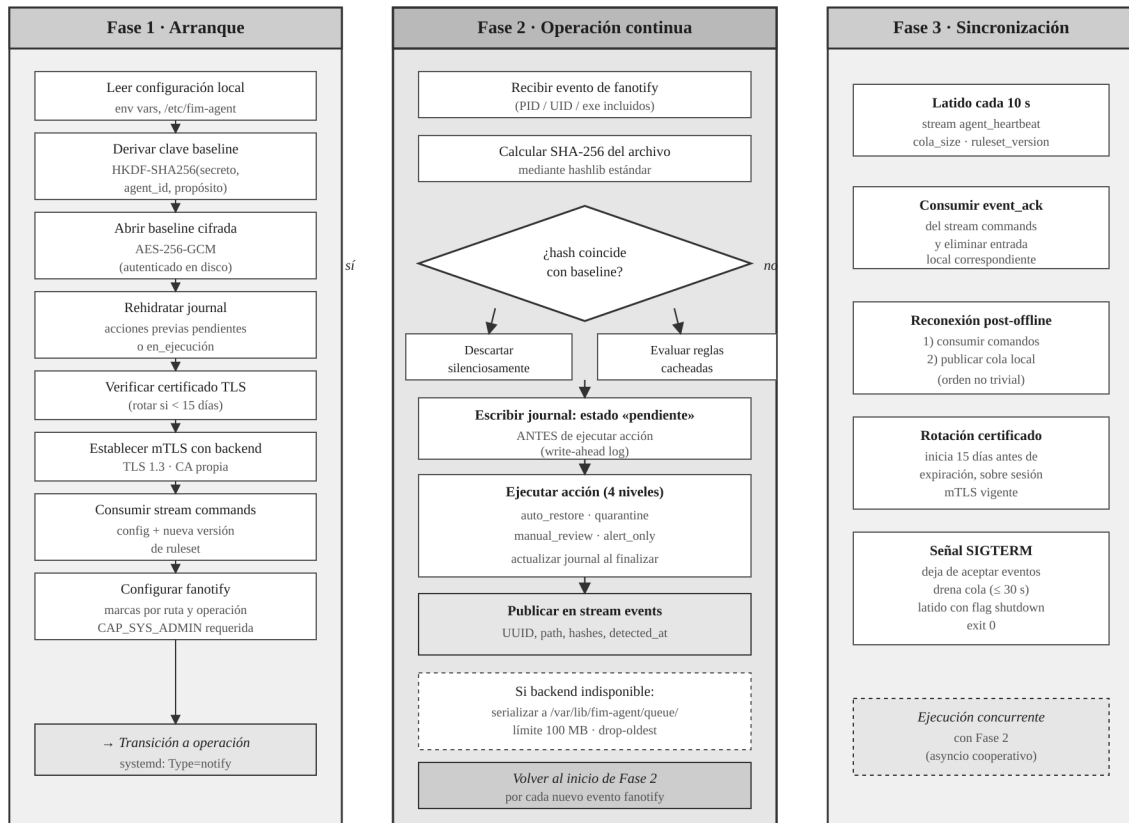


Figura 3. Flujo de procesamiento del agente: tres fases de operación

Nota. La fase de arranque es secuencial. Las fases 2 y 3 ejecutan concurrentemente mediante primitivas asíncrono cooperativas. El registro de acciones previas con escritura anticipada (journal) protege la integridad operacional ante crashes del proceso.

Al arrancar, el agente rehidrata el registro de acciones previas: revisa el directorio journal en busca de entradas marcadas como pendientes o en ejecución, que corresponden a acciones iniciadas pero no completadas en ejecuciones anteriores por crash o reinicio del proceso. Cada entrada encontrada se reintenta o se reporta al backend según corresponda. A continuación, el agente recupera su configuración operativa y la lista actualizada de reglas mediante el stream de comandos, verifica la vigencia de su certificado TLS y establece la conexión mutua con el backend.

Durante la operación normal, el agente escucha los eventos de fanotify sobre los paths configurados. Para cada evento de modificación recibido, el agente calcula el hash SHA-256 del archivo actualizado, consulta la baseline cifrada para obtener el hash de referencia, y determina si se trata de un cambio real. Si el hash coincide con la baseline, el evento se descarta silenciosamente. Si difiere, el agente evalúa las reglas locales para determinar la acción correspondiente entre las cuatro disponibles: restauración automática, cuarentena,

revisión manual o solo alerta. Antes de ejecutar cualquier acción modificatoria, el agente escribe una entrada de registro de acción previa con estado pendiente, persistida en disco para sobrevivir a crashes. Al completar la acción, la entrada se actualiza a estado completado o fallido con los detalles del resultado.

El agente publica el evento resultante en el stream events de Valkey, incluyendo su identificador único UUID, el path afectado, el hash detectado, el estado asignado, un timestamp detected_at, la versión del esquema de mensajes y una serie de campos contextuales. Si el backend no se encuentra disponible, el evento se serializa a un archivo JSON en el directorio de cola local, cuya capacidad está acotada a cien megabytes con política drop-oldest: cuando se supera el límite, los archivos más antiguos se descartan con registro de warning. Cuando la ocupación de la cola supera el ochenta por ciento, el próximo latido emitido al reconectar incluye un indicador queue_pressure que la interfaz administrativa traduce en un banner operativo.

El comportamiento al reconectar merece detalle particular. El agente no se limita a vaciar la cola pendiente; primero consume todos los comandos acumulados en el stream commands, y recién después publica los eventos encolados. Esta inversión de orden, aparentemente menor, resulta crítica: garantiza que los eventos acumulados durante la desconexión se persistan en el backend evaluados contra el ruleset correcto, y no contra reglas desactualizadas que habrían estado vigentes al momento original de la detección pero que el administrador modificó durante la ventana de desconexión. Complementariamente, cada comando de actualización de baseline o sincronización de reglas incluye una versión monotónicamente creciente que el agente persiste localmente y utiliza para descartar mensajes con versión menor a la última aplicada, obteniendo así idempotencia ante re-entregas eventuales de Valkey.

El agente emite un latido al stream agent_heartbeat cada diez segundos, conteniendo su identificador, un timestamp, el tamaño de la cola local y la versión de ruleset vigente. Si el agente recibe la señal SIGTERM de terminación graceful, deja de aceptar nuevos eventos, drena la cola local publicándola al stream events con un timeout de treinta segundos, y finaliza con código de salida cero. Durante el drenaje, el latido incluye un indicador shutdown que permite al backend mostrar el agente como draining en la interfaz administrativa. La Tabla 6 sintetiza las políticas de resiliencia del agente.

Tabla 6. Políticas de resiliencia del agente

Situación	Política adoptada
Crash mid-action	Registro de acción previa rehidratado al arranque y reintentado o reportado.
Desconexión del backend	Cola local acotada a 100 MB con drop-oldest y banner al reconectar si la presión superó 80 %.
Reconexión al backend	Comandos pendientes procesados antes que eventos encolados para evaluación con ruleset correcto.
Recepción de comando duplicado	Descarte por versión de ruleset menor a la última aplicada.
Recepción de SIGTERM	Drenaje graceful con timeout de 30 s y latido con indicador shutdown.
Compromiso del volumen sin la clave maestra	Baseline ilegible: cifrada con AES-256-GCM y clave derivada HKDF fuera del volumen.

4.4 Motor de decisión, cadena superseded y protocolo ACK

El motor de decisión local constituye una contribución conceptual central del diseño y diferencia al sistema de soluciones que se limitan a emitir alertas sin tomar decisiones autónomas. El motor opera con cuatro niveles de acción. El nivel restauración automática se aplica a archivos de máxima criticidad, típicamente binarios del sistema o archivos de configuración de autenticación; el agente restaura automáticamente desde baseline, verifica el hash posterior y emite el evento con estado `auto_restored`. El nivel cuarentena aplica a archivos sospechosos que requieren análisis forense; el agente mueve el archivo al directorio de cuarentena con permisos restringidos y emite el evento con estado `quarantined`. El nivel revisión manual aplica a archivos cuyos cambios requieren juicio experto; el agente no toca el archivo y emite el evento con estado `pending`. El nivel solo alerta opera como comportamiento por defecto cuando ninguna regla `matchea`; el evento se emite con estado `alert_only` para fines de auditoría.

La gestión de cadenas de eventos con estado `superseded` resuelve una condición de carrera estructural de los flujos de aprobación humana. La situación problemática es la siguiente: un archivo monitoreado cambia, generando un evento `pending`; antes de que el administrador lo resuelva, el archivo cambia nuevamente. La resolución del diseño consiste en que, ante el nuevo cambio, el agente busca el evento `pending` existente para el mismo `path`, lo marca como `superseded`, y crea un nuevo evento `pending` con referencia al anterior mediante `parent_event_id`. Solo el último evento de la cadena permanece visible como

pending en la interfaz administrativa. Cuando el administrador eventualmente aprueba el último evento, el backend hasha el archivo actual del filesystem, y no el hash registrado en el evento, para actualizar la baseline, garantizando así que el estado aprobado corresponde al estado presente.

El protocolo de entrega exactamente-una-vez extremo a extremo complementa la cadena superseded y resuelve un problema distinto: la combinación de garantías at-least-once de Valkey con la posibilidad de crashes en cualquier punto del camino entre agente y backend, que sin controles adicionales llevaría a pérdidas o duplicaciones. La Figura 4 presenta el diagrama de secuencia del protocolo.

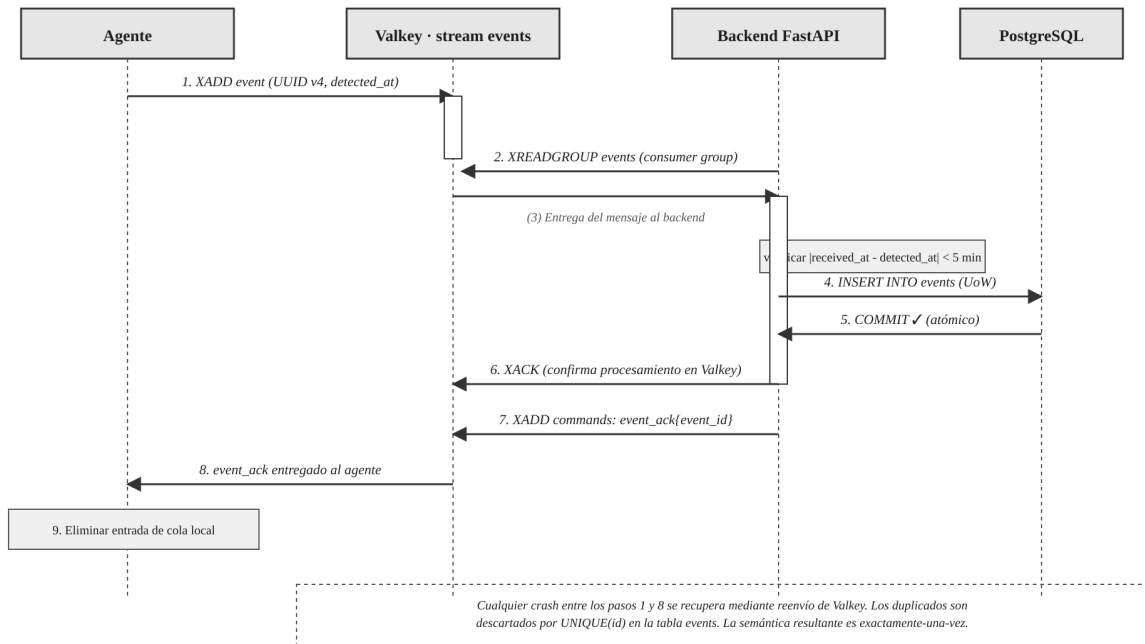


Figura 4. Diagrama de secuencia del protocolo de entrega exactamente-una-vez

Nota. La combinación de UUID v4 generado en origen, XACK explícito al consumir, publicación de event_ack y eliminación de la entrada en la cola local del agente garantiza la semántica exactamente-una-vez extremo a extremo sobre una capa de mensajería con garantías at-least-once.

El flujo es el siguiente. El agente genera un identificador UUID versión 4 al crear cada evento. El backend consume el stream events mediante un consumer group. Tras persistir el evento en PostgreSQL, el backend ejecuta la confirmación XACK sobre el stream, y a continuación publica un mensaje event_ack en el stream commands con el identificador del evento confirmado. El agente, al recibir el event_ack, elimina la entrada correspondiente de su cola local. Cualquier crash intermedio se recupera mediante reenvío, y cualquier duplicación

eventual se descarta por restricción de unicidad sobre el identificador del evento en la tabla events.

4.5 Flujo del backend: lifecycle formal, bloqueo optimista y anti-replay

El backend estructura sus módulos por dominio funcional: autenticación, eventos, reglas, acciones, alertas, agentes, notificaciones, salud del sistema y auditoría. Cada dominio sigue la organización en capas descripta en el apartado 4.9. La inicialización del esquema de base de datos se delega a un contenedor separado db-init que ejecuta la creación de tablas y el seed del primer administrador, tras lo cual termina. El backend propiamente dicho arranca solo después de que db-init haya completado, lo que permite una inicialización del servicio propiamente dicho en pocos segundos en lugar de quince a veinte.

Tres decisiones de auditoría resultan centrales para el backend. La primera es la máquina de estados explícita del evento, que la Figura 5 representa gráficamente. Cualquier transición no enumerada en el grafo se considera inválida y el servicio la rechaza con código HTTP 409. La implementación se materializa en un validador de transición y se acompaña de pruebas unitarias que cubren cada transición válida e inválida.

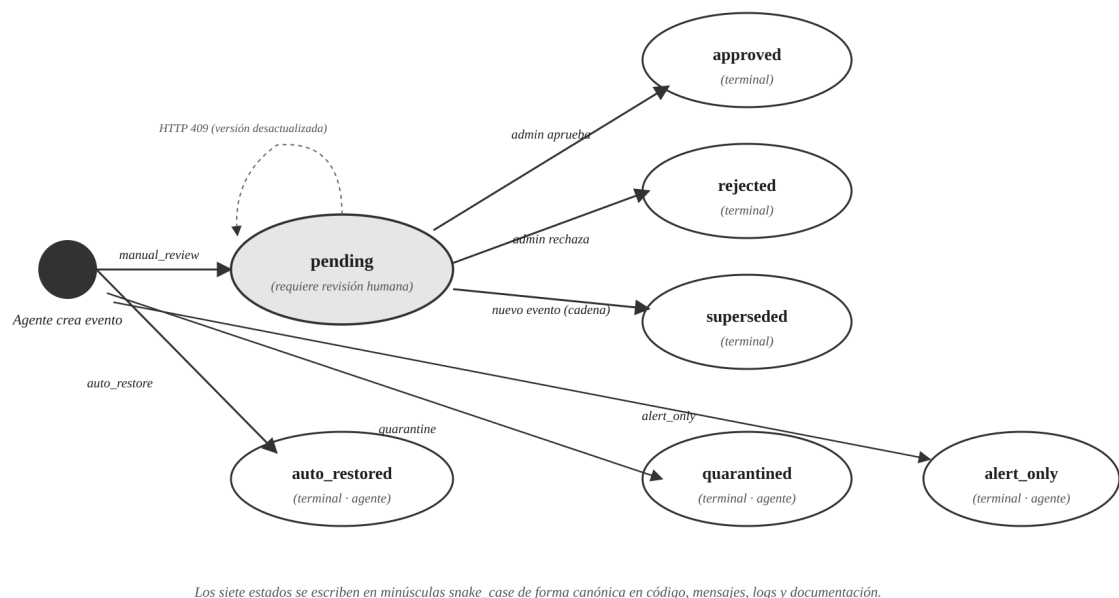


Figura 5. Máquina de estados explícita del evento (siete estados posibles)

Nota. Los estados se escriben en minúsculas *snake_case* de forma canónica en todo el sistema. El estado *pending* admite tres transiciones de salida (*approved*, *rejected*, *superseded*). Los estados *auto_restored*, *quarantined* y *alert_only* son terminales y creados directamente por el agente sin intervención humana.

Tabla 7. Máquina de estados explícita del evento

Estado destino	Transiciones de entrada permitidas	Transiciones de salida permitidas
pending	Creación por agente con acción <i>manual_review</i> .	<i>approved</i> , <i>rejected</i> , <i>superseded</i>
approved	Desde <i>pending</i> , por aprobación del administrador.	(terminal)
rejected	Desde <i>pending</i> , por rechazo del administrador.	(terminal)
superseded	Desde <i>pending</i> , por cadena o por <i>re-scan</i> .	(terminal)
auto_restored	Creación por agente con acción <i>auto_restore</i> .	(terminal)
quarantined	Creación por agente con acción <i>quarantine</i> .	(terminal)
alert_only	Creación por agente con acción <i>alert_only</i> o por defecto.	(terminal)

Nota. Los siete estados se escriben en minúsculas *snake_case* de forma canónica en todo el sistema: código, mensajes, logs y documentación.

La segunda decisión es el bloqueo optimista sobre la tabla de eventos. La columna *version*, entera por defecto con valor inicial cero, se incrementa atómicamente en cada actualización de estado mediante una cláusula SQL con WHERE que verifica simultáneamente el identificador, la versión esperada y el estado previo. Si el UPDATE afecta cero filas, el servicio retorna HTTP 409 con cuerpo que identifica el conflicto como resolución previa o como *superseded* automático. El frontend traduce esta respuesta en un mensaje al operador y refresca automáticamente la vista. Esta decisión evita condiciones de carrera entre dos administradores aprobando simultáneamente el mismo evento, así como entre la aprobación manual y la cadena *superseded* activada por un nuevo evento del agente durante la ventana de decisión humana.

La tercera decisión es la detección de desincronización horaria con fines anti-replay. Todo evento publicado por el agente incluye un timestamp *detected_at*; el backend agrega

received_at al consumirlo. Si el valor absoluto de la diferencia entre ambos supera cinco minutos, el backend rechaza el evento con código clock_skew y lo persiste en la tabla rejected_events_audit para investigación posterior. Esta decisión detecta tanto relojes desincronizados, problema operativo habitual, como intentos de reinyección de eventos antiguos, amenaza de seguridad. Complementariamente, todo mensaje en los streams events y commands incluye una versión de esquema entera; el backend rechaza mensajes con esquema mayor al soportado con código UNSUPPORTED_SCHEMA, mientras que el agente ignora campos desconocidos para habilitar compatibilidad hacia adelante.

4.6 Frontend administrativo y hardening web

La aplicación frontend se implementa como Single Page Application en React 19 con TypeScript, empaquetada con Vite, con estilos Tailwind CSS 4.2 bajo integración nativa mediante @tailwindcss/vite, con gestión de estado del servidor mediante TanStack Query en su versión 5, con estado global del cliente mediante Zustand, con cliente HTTP Axios e interceptores para la gestión del token JWT, con enrutamiento mediante React Router en su versión 7 y con el gestor de paquetes pnpm. La organización del código sigue una estructura de cuatro carpetas principales: api con cliente y endpoints agrupados por dominio, stores con tiendas Zustand, pages con componentes de página, y components subdividida en layout y ui.

El backlog funcional comprende treinta y una historias de usuario, conformadas por veinticuatro originales más siete incorporadas tras la auditoría interna del proyecto. Las funcionalidades cubren autenticación, visualización de eventos con filtrado y paginación, aprobación y rechazo individual o en lote, gestión de reglas, visualización de alertas en tiempo real mediante SSE (Server-Sent Events), gestión de agentes, reintento de notificaciones fallidas, y cambio forzado de contraseña en primer inicio de sesión.

Cuatro decisiones de hardening resultan especialmente relevantes. La primera es el conjunto de cabeceras HTTP emitidas por nginx al servir el frontend, que incluye una política de seguridad de contenido restrictiva que limita el origen de scripts y estilos, una política estricta de transporte seguro, la cabecera X-Frame-Options con valor DENY para prevenir inclusión en frames externos, la validación de la cabecera Origin contra una lista blanca en el backend, y cookies con atributo SameSite configurado como Strict. Esta configuración mitiga conjuntamente los ataques de cross-site scripting, clickjacking y cross-site request forgery.

La segunda decisión es el almacenamiento seguro de tokens en el cliente: el access token vive exclusivamente en memoria dentro de la store Zustand, nunca en localStorage ni sessionStorage, y el refresh token se transmite como cookie httpOnly con atributos Secure, SameSite Strict y path restringido al endpoint de renovación. Esta combinación resiste tanto XSS como CSRF. La tercera es el uso exclusivo de una biblioteca de visualización de diferencias con escapado activo en el componente DiffViewer, con prohibición absoluta de inyección HTML sin sanitización en todo el código mediante una regla de ESLint. La cuarta es el flujo de cambio forzado de contraseña en el primer inicio de sesión: el seed inicial del administrador crea el usuario con un flag que obliga al cambio; el primer login retorna un token con alcance restringido exclusivamente al cambio de contraseña, y el frontend bloquea cualquier otra navegación hasta completar el cambio.

4.7 Orquestación de notificaciones con fallbacks automáticos

La integración con el motor de notificaciones se realiza mediante un componente interno denominado NotificationDispatcher, que desacopla al código de negocio del mecanismo concreto de entrega. Este componente opera con un motor principal configurable, por defecto n8n, y una cadena ordenada de alternativas de respaldo que se activan automáticamente ante indisponibilidad del principal. La disponibilidad de n8n se consulta mediante un healthcheck activo con caché de diez segundos para evitar sobrecarga.

La Figura 6 presenta el diagrama de decisión del componente, incluyendo el camino principal, la cascada de fallbacks y la persistencia final en la cola de mensajes fallidos.

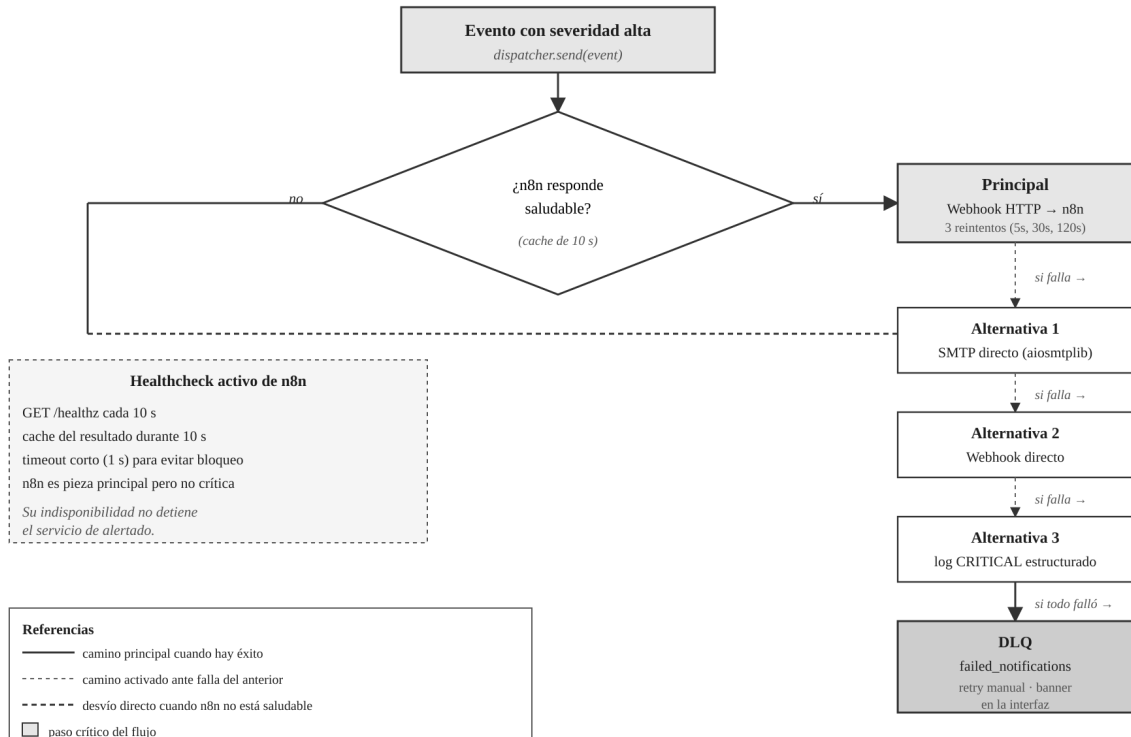


Figura 6. NotificationDispatcher: cascada de fallbacks automáticos

Nota. El healthcheck activo evalúa la salud de n8n cada diez segundos. La cascada asegura continuidad del servicio de alertado aun ante indisponibilidad del orquestador principal. La persistencia final en *failed_notifications* habilita reintento manual desde la interfaz administrativa.

El flujo operativo es el siguiente. Cuando un evento con severidad crítica o alta requiere notificación, el dispatcher primero verifica la salud del motor principal. Si n8n responde saludable, el dispatcher envía el webhook HTTP con payload JSON hacia el endpoint configurado de n8n, que ejecuta el flujo de trabajo y reencamina la notificación hacia los canales configurados por el administrador. Si n8n no responde saludable, o si el envío al endpoint falla, el dispatcher recorre la cadena de alternativas en orden preestablecido. La primera alternativa típica es el envío directo por SMTP mediante la biblioteca aiosmtplib, con las credenciales de un servidor de correo saliente configuradas en variables de entorno. La segunda alternativa es el envío directo de webhook hacia una plataforma de mensajería corporativa con la URL configurada también en entorno. La última alternativa es el logging estructurado con nivel crítico, como último esfuerzo antes de perder la alerta.

Si todas las alternativas fallan, el mensaje se persiste en la tabla *failed_notifications* con los campos identificador del evento, payload, último error registrado, timestamp de fallo

y cantidad de reintentos. La interfaz administrativa muestra un banner persistente mientras la tabla contenga al menos una fila con al menos tres reintentos fallidos, y una vista dedicada permite al operador reintentar individualmente, descartar o reintentar en lote todas las notificaciones pendientes. Las notificaciones hacia n8n emplean internamente un esquema de reintento exponencial de tres intentos con retardos de cinco, treinta y ciento veinte segundos antes de activar la cadena de fallbacks.

Esta arquitectura convierte a n8n en pieza principal sin hacerla crítica: aprovecha su ecosistema de integraciones visuales y su flexibilidad cuando opera saludable, pero asegura continuidad del servicio cuando no lo hace. La Tabla 8 sintetiza el orden de precedencia.

Tabla 8. Orden de precedencia para entrega de notificaciones

Prioridad	Mecanismo	Activación
Principal	Webhook HTTP hacia n8n	n8n saludable según healthcheck de 10 s.
Alternativa 1	SMTP directo	n8n indisponible o envío fallido tras reintentos.
Alternativa 2	Webhook directo a plataforma de mensajería	Alternativa 1 no configurada o fallida.
Alternativa 3	Log estructurado con nivel crítico	Último esfuerzo previo a persistencia en DLQ.
Persistencia final	Tabla failed_notifications	Todas las alternativas anteriores fallaron.

4.8 Seguridad criptográfica del sistema

La arquitectura criptográfica del sistema se articula en cinco capas complementarias, cada una con propósito específico y composición explícita con las demás. La primera capa es la autenticación mutua TLS del agente contra el backend, con autoridad certificadora propia gestionada por el propio backend. El flujo de bootstrap opera de la siguiente manera. El administrador pre-registra desde la interfaz administrativa un identificador de agente y un secreto de bootstrap de treinta y dos bytes aleatorios, que se almacena hasheado en la base de datos. El agente, al arrancar por primera vez, genera su par de claves y envía al endpoint de bootstrap una solicitud de firma de certificado firmada con el secreto compartido mediante HMAC, junto con su identificador. El backend verifica el HMAC, firma el CSR, emite un

certificado válido por noventa días y lo retorna. El agente persiste el certificado en su directorio local. La rotación se inicia quince días antes de la expiración mediante una renovación realizada sobre la sesión TLS mutuo existente. La revocación se gestiona mediante la tabla `revoked_certificates`, que el backend verifica en cada establecimiento de handshake con caché de un minuto para evitar sobrecarga.

La segunda capa es la firma HMAC-SHA256 de comandos emitidos por el backend hacia el agente. Cada comando publicado en el stream `commands` incluye un campo `signature` que corresponde al HMAC computado sobre una representación JSON canónica del payload, empleando un secreto compartido específico para cada agente que se genera y distribuye durante el bootstrap. El agente rechaza cualquier comando con firma inválida. Esta capa materializa una defensa en profundidad sobre la anterior: incluso si un atacante comprometiera el motor Valkey sin comprometer el certificado mTLS del agente, no podría inyectar comandos válidos que el agente ejecutara.

La tercera capa es la gestión de tokens JWT para los administradores humanos. El `access token` tiene expiración de quince minutos y el `refresh token` de siete días, con rotación en cada uso: el refresh anterior se invalida al emitir uno nuevo. Los identificadores `jti` de tokens revocados se mantienen en una lista negra en Valkey con tiempo de vida igual al tiempo restante de validez del token; cuando el token expira naturalmente, la entrada en la lista negra se borra automáticamente por el propio TTL, evitando crecimiento indefinido. La firma emplea un esquema multi-clave mediante variables de entorno: se firma con la clave actual y se verifica intentando primero la actual y luego la previa; al rotar, la actual se promueve a previa y se genera una nueva actual. Este esquema permite rotación de clave sin caída del servicio y revocación inmediata de refresh tokens comprometidos.

La cuarta capa es el hashing de contraseñas de operadores con Argon2id. La configuración se instancia como instancia única en el módulo `core` de seguridad, con parámetros configurables mediante variables de entorno. Los valores por defecto recomendados son `time_cost` de tres iteraciones, `memory_cost` de 65.536 kibibytes equivalentes a sesenta y cuatro mebibytes y `parallelism` de cuatro hilos, calibrados para hardware de servidor estándar con al menos cuatro núcleos y ocho gibibytes de memoria disponible. Al arrancar, el backend ejecuta un benchmark interno con los parámetros activos y emite un warning si el tiempo observado resulta menor a doscientos cincuenta milisegundos, indicando parámetros demasiado débiles, o mayor a dos segundos, indicando parámetros

excesivos que penalizarán la experiencia de login. El rango objetivo es de quinientos milisegundos a un segundo, conforme a las recomendaciones actuales para funciones de hashing de contraseñas.

La quinta capa es el cifrado autenticado de la baseline en reposo, descrito en los apartados 2.10 y 4.3. La Tabla 9 resume la distribución de controles criptográficos.

Tabla 9. Distribución de controles criptográficos

Propósito	Primitiva	Ubicación
Autenticación del canal agente-backend	mTLS sobre TLS 1.3 con CA propia	Handshake de cada conexión.
Autenticación del mensaje backend-agente	HMAC-SHA256 con secreto compartido	Campo signature de cada comando.
Autenticación del usuario administrativo	JWT multi-clave con lista negra jti	Cabecera Authorization Bearer.
Hashing de contraseñas de operadores	Argon2id parametrizable con benchmark	Columna users.password_hash.
Cifrado de baseline en reposo	AES-256-GCM con clave HKDF-SHA256	/opt/fim-agent/storage/baseline/.
Integridad de eventos en tránsito	TLS 1.3 (canal) + XACK bidireccional (entrega)	Streams Valkey.
Detección de replay	Timestamps dobles con umbral 5 minutos	Campos detected_at y received_at.

4.9 Organización del código: arquitectura por capas

El backend adopta una organización en capas inspirada en los principios de Arquitectura Limpia, con el propósito de aislar la lógica de negocio de las dependencias tecnológicas concretas. La capa de presentación está conformada por los routers de FastAPI, que se limitan a decodificar la solicitud, invocar el caso de uso correspondiente y serializar la respuesta. La capa de aplicación contiene los casos de uso, cada uno de los cuales implementa una operación de negocio completa y orquesta las interacciones con los repositorios a través de la Unit of Work. La capa de dominio modela las entidades de negocio (Event, Baseline, Rule, User, AuditEntry) y las reglas puras que las gobiernan, sin dependencias externas. La capa de infraestructura provee las implementaciones concretas: repositorios sobre

PostgreSQL, publicadores sobre Valkey, clientes HTTP para n8n y para los fallbacks, y adaptadores criptográficos.

La adopción conjunta de los patrones Repository y Unit of Work estructura el acceso a datos con tres ventajas operativas concretas. La primera ventaja consiste en que los casos de uso pueden ser verificados mediante pruebas unitarias con repositorios simulados en memoria, sin necesidad de levantar una base de datos real, lo que permite tiempos de ejecución de pruebas en milisegundos en lugar de segundos. La segunda ventaja consiste en que el Unit of Work gestiona el ciclo de vida de la transacción de base de datos mediante un contexto gestionado: al ingresar adquiere una conexión del pool pre-calentado y abre una transacción; al salir correctamente confirma la transacción y devuelve la conexión; al salir por excepción revierte automáticamente la transacción y devuelve la conexión. Con esta disciplina, la categoría de errores en la cual un desarrollador olvida una reversión explícita tras un error intermedio se elimina por construcción. La tercera ventaja consiste en que un cambio futuro del motor de base de datos requeriría modificar solo la capa de infraestructura sin impactar el resto del código.

4.10 Contenedorización del servidor central y despliegue del agente

La contenedorización mediante Docker Compose aplica exclusivamente al servidor central. El archivo `docker-compose.yml` define los servicios `db`, `valkey`, `db-init`, `backend`, `frontend` y `n8n`, cada uno con sus `healthchecks`, políticas de reinicio, redes aisladas según ámbito funcional, y volúmenes persistentes donde corresponda. El servicio `db-init` corre una sola vez al arranque del stack, ejecuta la creación del esquema mediante `SQLModel` y el `seed` del primer administrador, y finaliza. El servicio `backend` arranca solo tras la finalización exitosa de `db-init`, lo que simplifica el razonamiento sobre el estado del sistema durante el arranque.

Los agentes, en cambio, se despliegan como servicios nativos del sistema operativo en cada anfitrión monitoreado. Esta decisión responde a cuatro razones técnicas convergentes. La primera razón es la necesidad de visibilidad directa sobre el sistema de archivos del anfitrión: desde un contenedor, el agente vería únicamente el sistema de archivos del propio contenedor, que nadie modificaría. La segunda razón es la necesidad de la capacidad `CAP_SYS_ADMIN` del núcleo para operar `fanotify`, cuya concesión a un contenedor anula en la práctica el aislamiento que los contenedores pretenden proveer. La tercera razón es que el ciclo de vida

del agente está acoplado al del anfitrión y no a un orquestador de contenedores: si el anfitrión reinicia, el agente debe reiniciarse con él, gestionado por systemd. La cuarta razón es que el monitoreo de binarios del sistema requiere ver los binarios reales del anfitrión, no los del contenedor. La instalación del agente se realiza mediante un script que despliega un entorno virtual Python en `/opt/fim-agent`, crea un usuario del sistema `fim-agent` con shell deshabilitada, configura los permisos restrictivos sobre el directorio de secretos, registra el unit file de systemd e inicia el servicio. Extractos representativos del unit file se incluyen en el Anexo E.

4.11 Operaciones y observabilidad

Cinco decisiones operativas complementan el diseño funcional del sistema. La primera decisión es la limitación de tasa multicapa implementada en Valkey mediante contadores con TTL: el endpoint de login permite cinco intentos cada quince minutos por combinación usuario-IP; la API autenticada permite cien solicitudes por minuto por usuario; el consumo del stream de eventos se limita a cien eventos por minuto por agente. La segunda decisión es el logging estructurado en formato JSON mediante structlog con un middleware de sanitización que filtra los campos sensibles (contraseñas, tokens de acceso, tokens de refresco, secretos de bootstrap, secreto maestro de baseline, firmas) antes de emitir, y con retención de treinta días. El contenido de los diffs de archivos está explícitamente prohibido en los logs; solo se registran hashes anterior y nuevo junto con diferencia de tamaño.

La tercera decisión es la provisión de un endpoint de salud por componente que retorna el estado de PostgreSQL, Valkey, n8n y cada agente conocido, con tres valores posibles por componente: `ok`, `degraded` o `down`. El frontend consulta este endpoint cada diez segundos y, ante cualquier componente no `ok`, muestra un banner persistente en la cabecera. La cuarta decisión es el esquema de latido y estado de agente: el agente publica un latido cada diez segundos; el backend marca un agente como `offline` si no recibe latido por treinta segundos, y como `dead` tras cinco minutos sin latido, lo que dispara adicionalmente un webhook n8n para alerta inmediata al equipo operativo. La quinta decisión es el registro diligente en la tabla `audit_log` con retención ilimitada de todas las acciones administrativas sensibles, invocado como efecto lateral de los servicios correspondientes: inicio y cierre de sesión, operaciones CRUD sobre reglas, aprobaciones y rechazos, re-scans, cambios de configuración, y finalización graceful de agentes.

4.12 Estado real del proyecto

Por razones de honestidad metodológica y conforme al principio de transparencia académica que orienta el presente trabajo, corresponde declarar que el proyecto se encuentra en fase de documentación y diseño detallado. El documento de arquitectura del proyecto y el registro de decisiones de auditoría constituyen artefactos formales y completos de especificación. La Tabla 10 sintetiza el estado por componente.

Tabla 10. Estado actual del proyecto por componente

Componente	Estado	Siguiente actividad
Documento de arquitectura	Completado	—
Registro de decisiones de auditoría	Completado	—
Backlog funcional (31 historias de usuario)	Completado	—
Modelo de datos y diagrama E/R	Diseñado	Script SQL definitivo.
Agente (Python)	No implementado	Codificación completa.
Backend (FastAPI)	No implementado	Codificación completa.
Frontend (React)	No implementado	Codificación completa.
Flujos n8n	Definidos	Importación y configuración.
PKI propia y certificados mTLS	Definido	Generación y prueba de bootstrap.
Docker Compose	Estructurado	Healthchecks y secrets definitivos.
Unit file de systemd del agente	Redactado	Empaquetado en script de instalación.
Protocolo experimental	Completado	Ejecución sobre laboratorio definitivo.

Nota. La ejecución del protocolo experimental y la captura de datos sobre el laboratorio definitivo constituyen la fase inmediata siguiente del proyecto.

CAPÍTULO 5. RESULTADOS

«En Dios confiamos. Los demás deben aportar datos.»

— Atribuida a W. Edwards Deming

En el presente capítulo se presentan los protocolos experimentales definitivos y las planillas de captura diseñadas para documentar las mediciones resultantes. La fase de implementación y la ejecución efectiva del protocolo sobre el laboratorio definitivo constituyen la fase inmediata siguiente del proyecto, y sus resultados numéricos se incorporarán mediante una adenda formal una vez completada dicha fase, conforme al compromiso explícito asumido en el Capítulo 7 y en el Anexo F.

5.1 Consideración metodológica sobre la presentación de resultados

Se optó deliberadamente por presentar los protocolos y las planillas en lugar de resultados numéricos derivados de simulaciones o ejecuciones parciales, por tres razones metodológicas convergentes que conviene explicitar ante la comisión evaluadora. La primera razón es que el proyecto se encuentra en fase de diseño detallado; la validación empírica rigurosa debe realizarse sobre la infraestructura definitiva una vez completada la implementación de todos los componentes, no sobre prototipos parciales cuya ejecución podría sesgar las mediciones y comprometer la validez interna del estudio. La segunda razón es que la publicación de valores numéricos sin respaldo experimental verificable constituiría una forma de hipotetización posterior a los resultados que compromete la validez metodológica del trabajo. La tercera razón es que la publicación de planillas junto con los protocolos detallados habilita la replicación del experimento por parte de terceros, lo cual constituye en sí mismo una contribución académica de valor incluso antes de la ejecución por parte de los propios autores.

En concordancia con esta decisión, los autores asumen el compromiso explícito de publicar una adenda metodológica formal tras la ejecución del protocolo sobre el laboratorio definitivo. Dicha adenda completará las planillas presentadas en este capítulo con los valores empíricos capturados, confirmará o refutará la hipótesis mediante comparación contra los umbrales preestablecidos a priori en el Capítulo 1, y actualizará las secciones pertinentes del

Capítulo 6 (Discusión) y del Capítulo 7 (Conclusiones). Esta secuencia metodológica preserva la integridad del compromiso falsacionista que caracteriza al diseño experimental riguroso.

5.2 Protocolo de medición de latencia de detección

La primera batería mide la latencia de detección, definida como el intervalo entre el instante en que se completa la modificación de un archivo monitoreado y el instante en que el evento correspondiente es recibido por el backend. El protocolo ejecuta el generador de carga con quinientas modificaciones distribuidas uniformemente sobre una ventana de treinta minutos, con tamaños de archivo aleatorios entre un kilobyte y un megabyte, y con distribución de operaciones consistente en setenta por ciento de modificación, veinte por ciento de creación y diez por ciento de eliminación. Cada operación se registra con timestamp de nanosegundos y se correlaciona con el timestamp de recepción en el backend.

Tabla 11. Planilla de captura — Latencia de detección

Estadístico	Valor observado (ms)	Umbral	Cumple
Media aritmética	[pendiente]	—	
Mediana	[pendiente]	—	
Desvío estándar	[pendiente]	—	
Valor mínimo	[pendiente]	—	
Valor máximo	[pendiente]	—	
Percentil 95	[pendiente]	—	
Percentil 99 (indicador principal)	[pendiente]	< 1.000	
Eventos medidos	[pendiente]	500	
Eventos perdidos	[pendiente]	0	
Eventos rechazados por desincronización horaria	[pendiente]	0	

Nota. Planilla a completar durante la ejecución del protocolo. Los valores consignados como «[pendiente]» corresponden al estado actual del proyecto y se sustituirán con los valores empíricos capturados en la adenda metodológica.

5.3 Protocolo de medición de tiempo de notificación

La segunda batería mide el tiempo transcurrido entre la recepción de un evento por el backend y la emisión exitosa de la notificación final, bajo tres escenarios de concurrencia. Se realizan mil invocaciones secuenciales por escenario. Cada invocación se instrumenta en los puntos críticos del flujo: recepción, consulta a baseline, persistencia con bloqueo optimista, evaluación de severidad y emisión del webhook.

Tabla 12. Planilla de captura — Tiempo de notificación

Escenario	Media (ms)	P50 (ms)	P95 (ms)	P99 (ms)	Cumple
Secuencial (1 concurrente)	[pendiente]	[pendiente]	[pendiente]	[pendiente]	
Moderada (50 concurrentes)	[pendiente]	[pendiente]	[pendiente]	[pendiente]	
Alta (100 concurrentes)	[pendiente]	[pendiente]	[pendiente]	[pendiente]	

Nota. Umbral de aceptación: percentil 99 menor a 5.000 ms en los tres escenarios.

5.4 Protocolo de cobertura de automatización del triage

La tercera batería evalúa la cobertura de automatización mediante análisis contra la checklist derivada de NIST SP 800-61r2. Cada tarea se clasifica como automatizada, automatizable pero manual por diseño, o necesariamente manual por requerir juicio experto.

Tabla 13. Planilla de captura — Cobertura de automatización del triage

N.	Tarea NIST SP 800-61r2	Automatizada	Requiere humano
1	Recepción del evento inicial con metadatos.	[]	[]
2	Cálculo del hash actualizado del archivo.	[]	[]
3	Comparación contra la baseline vigente.	[]	[]
4	Identificación del tipo de cambio.	[]	[]
5	Consulta de reglas aplicables al path.	[]	[]
6	Asignación de severidad según regla.	[]	[]
7	Selección de acción automática.	[]	[]
8	Ejecución de la acción automática.	[]	[]

N.	Tarea NIST SP 800-61r2	Automatizada	Requiere humano
9	Registro del evento y auditoría.	[]	[]
10	Actualización de eventos previos (superseded).	[]	[]
11	Notificación según severidad.	[]	[]
12	Discriminación entre cambio autorizado no registrado y cambio sospechoso.	[]	[]
13	Decisión sobre escalamiento.	[]	[]

Nota. Las tareas 12 y 13 requieren inherentemente juicio experto humano por diseño del sistema.

5.5 Validación funcional mediante casos de prueba

La batería funcional valida el comportamiento correcto del sistema ante los escenarios operativos del backlog. Se ejecuta una prueba por cada historia del backlog consolidado: las veinticuatro historias originales más las siete historias incorporadas tras la auditoría interna del proyecto. El criterio de aceptación exige aprobación de la totalidad de las treinta y una historias.

5.6 Protocolo de resiliencia offline

La batería evalúa la capacidad del agente de preservar eventos ante desconexiones del backend. Se detiene manualmente el servicio backend durante cinco minutos, durante los cuales el generador produce eventos a una tasa constante de diez eventos por segundo, totalizando tres mil eventos. Tras el reinicio se verifica la preservación íntegra (hasta el límite de cien megabytes con política drop-oldest), la entrega en orden FIFO estricto tras procesamiento previo de comandos pendientes, y la ausencia de duplicaciones gracias al protocolo con identificador único por evento.

Tabla 14. Planilla de captura — Resiliencia offline

Indicador	Valor observado	Umbral	Cumple
Duración de la desconexión	[pendiente]	300 s	
Eventos generados durante la desconexión	[pendiente]	3.000	

Indicador	Valor observado	Umbral	Cumple
Eventos preservados en cola local	[pendiente]	Igual a generados	
Eventos entregados tras reconexión	[pendiente]	Igual a generados	
Orden FIFO preservado	[pendiente]	Sí	
Duplicaciones detectadas	[pendiente]	0	
Comandos procesados antes que eventos encolados	[pendiente]	Sí	
Tiempo de recuperación total	[pendiente]	< 30 s	

5.7 Comparación con grupo de control

La batería final compara directamente el grupo experimental contra el grupo de control. Ambos se ejecutan simultáneamente sobre la misma carga de trabajo y sobre los mismos directorios monitoreados. La ejecución simultánea aísla el efecto del diseño arquitectónico sobre la carga común y elimina la variabilidad proveniente de condiciones de sistema distintas.

Tabla 15. Planilla de captura — Comparación experimental vs. control

Indicador	Plataforma FIM	Script cron (15 min)	Factor de mejora
Mediana de latencia	[pendiente]	[pendiente]	[calcular]
Latencia percentil 99	[pendiente]	[pendiente]	[calcular]
Eventos perdidos	[pendiente]	[pendiente]	—
Baseline con separación y cifrado	Sí	No	—
Notificación automática con fallbacks	Sí	No	—
Trazabilidad de auditoría	Sí	No	—

5.8 Síntesis de criterios de aceptación

Tabla 16. Síntesis de criterios de aceptación a completar

Indicador	Valor observado	Umbral	Estado
Latencia de detección P99 (ms)	[pendiente]	< 1.000	
Tiempo de notificación P99 secuencial (ms)	[pendiente]	< 5.000	
Tiempo de notificación P99 con 100 concurrentes	[pendiente]	< 5.000	
Cobertura de automatización del triage (%)	[pendiente]	≥ 80	
Casos funcionales superados	[pendiente]	31 de 31	
Resiliencia offline (%)	[pendiente]	100	
Tiempo de recuperación (s)	[pendiente]	< 30	
Falsos negativos	[pendiente]	0	

Nota. La hipótesis se considerará confirmada si todos los indicadores alcanzan sus umbrales. Un solo indicador fallido obligará a refinar el diseño y repetir las baterías correspondientes.

CAPÍTULO 6. DISCUSIÓN

«No es suficiente con ser correcto; hay que ser convincente.»

— Adaptado de Sampieri, Fernández y Baptista (2014)

En el presente capítulo se desarrolla la discusión crítica de los resultados esperados a partir del protocolo especificado en el Capítulo 5, articulándolos con el marco teórico y contrastándolos con los antecedentes del estado del arte. Adicionalmente, se realiza un examen sistemático de las amenazas a la validez del estudio y se discute el posicionamiento de la propuesta respecto de tecnologías conceptualmente adyacentes.

6.1 Interpretación de los resultados esperados

Si los resultados del protocolo confirman los umbrales preestablecidos, el sistema habrá demostrado empíricamente que es posible alcanzar latencias sub-segundo mediante una arquitectura distribuida construida sobre componentes de código abierto. La compresión del tiempo medio de detección respecto del encuestamiento a quince minutos, utilizado por el grupo de control, tendría implicancias operativas directas sobre la ventana temporal disponible para los atacantes y configuraría una mejora de varios órdenes de magnitud. La cobertura de automatización del triage superior al ochenta por ciento representaría una liberación significativa de capacidad operativa, permitiendo a los analistas concentrarse en las dos fases que requieren juicio experto: la discriminación entre cambios legítimos no registrados y la decisión de escalamiento de incidentes.

6.2 Contribuciones diferenciales del diseño

Al comparar el diseño con las cuatro categorías caracterizadas en el apartado 2.5, se observan propiedades diferenciales relevantes. Frente a herramientas clásicas como Tripwire o AIDE, el sistema propuesto ofrece detección reactiva basada en eventos, baseline cifrada con separación arquitectónica, integración con canales externos y fallbacks automáticos, auditoría separada con retención ilimitada, y resolución formal de concurrencia mediante bloqueo optimista y cadena superseded. Frente a suites de detección de intrusiones a nivel de anfitrión como Wazuh, mantiene menor complejidad de despliegue a costa de menor amplitud funcional. Frente a soluciones comerciales, cede funcionalidad avanzada de aprendizaje

automático y correlación multifuente, pero ofrece accesibilidad económica y auditoría técnica completa del código.

Dos decisiones arquitectónicas merecen destacarse como contribuciones originales del trabajo. La primera es la gestión de cadena de eventos con estado superseded junto con el protocolo de entrega exactamente-una-vez y el bloqueo optimista, que configuran en conjunto un esquema formal de consistencia para flujos de aprobación humana en sistemas FIM, no documentado de manera unificada en la literatura consultada. La segunda es el esquema criptográfico en capas: autenticación mutua TLS con autoridad certificadora propia y rotación automática de certificados, firma HMAC de comandos, y cifrado autenticado de baseline con derivación de clave HKDF, que articula defensa en profundidad con una granularidad inusual en sistemas de código abierto comparables.

6.3 Adecuación al marco normativo argentino

La arquitectura contribuye al cumplimiento del artículo 9.º de la Ley 25.326 al proporcionar un mecanismo técnico de detección de adulteraciones y al preservar trazabilidad de las decisiones administrativas mediante la tabla de auditoría con retención ilimitada. Respecto de los cinco procesos de la Resolución 47/2018 de la AAIP, el sistema aporta principalmente al proceso de control de acceso mediante la tabla de auditoría, y al proceso de gestión de incidentes mediante detección temprana, notificación automatizada con fallbacks, y trazabilidad completa de la cadena de decisión. Respecto de la Segunda Estrategia Nacional de Ciberseguridad aprobada por la Resolución 44/2023, el diseño se alinea con los objetivos de fortalecer las capacidades de prevención, detección y respuesta.

No obstante, el sistema por sí solo no garantiza cumplimiento normativo integral: debe entenderse como componente técnico dentro de un programa más amplio de cumplimiento que incluya políticas organizacionales, procedimientos operativos, gestión de derechos de titulares, y eventualmente notificación de incidentes a la autoridad de aplicación correspondiente.

6.4 Amenazas a la validez interna

Se identifican cuatro amenazas a la validez interna del estudio que conviene documentar junto con sus mitigaciones previstas. La primera amenaza son los sesgos de

instrumentación, derivados del hecho de que las mediciones de tiempo se realizan mediante los propios componentes del sistema. Para mitigarla, el protocolo especifica mediciones redundantes mediante captura de paquetes con tcpdump y monitoreo de llamadas al sistema con strace, junto con verificación de consistencia cruzada. La segunda amenaza es la limitada variabilidad de las condiciones experimentales, mitigada parcialmente mediante documentación exhaustiva que habilita la replicación con otros parámetros. La tercera amenaza es la contaminación entre pruebas sucesivas por estado conservado en Valkey y PostgreSQL, mitigada mediante reinicialización controlada entre baterías. La cuarta amenaza es el efecto de maduración del sistema, mitigado mediante un período de calentamiento cuyas mediciones se descartan.

6.5 Amenazas a la validez externa

La primera amenaza a la validez externa es la limitada generalizabilidad a entornos con sistemas de archivos distribuidos, orquestadores de contenedores o volúmenes masivos, que no se encuentran dentro del alcance definido. La segunda amenaza es la evolución del panorama de amenazas: los resultados reflejarían capacidad frente a modificaciones que activan eventos fanotify, no frente a técnicas evasivas como manipulación a nivel de dispositivo de bloque, timestomping coordinado, o ataques de cadena de suministro del tipo del incidente CVE-2024-3094. La tercera amenaza es la especificidad del conjunto de directorios monitoreados: los criterios se validan sobre directorios críticos del sistema y subconjuntos de paths de aplicación, no sobre sistemas de archivos completos con decenas de miles de archivos. La amenaza residual principal, explicitada en el apartado 2.11, corresponde al compromiso del núcleo Linux por rootkits, cuya mitigación exige extender la raíz de confianza a nivel inferior.

6.6 Limitaciones tecnológicas del stack adoptado

El ejercicio de diseño detallado permitió identificar limitaciones tecnológicas específicas del stack adoptado que conviene documentar. En el dominio de detección de archivos, fanotify, a pesar de su madurez y sus ventajas sobre inotify, opera sobre el mismo modelo de notificación por descriptor de archivo; ante saturación del consumidor los eventos pueden perderse silenciosamente, por lo cual el agente debe monitorear activamente la condición y reportarla como anomalía. En el dominio de mensajería, Valkey depende de

persistencia periódica en disco mediante estrategia append-only por defecto, por lo que un crash intermedio puede perder los últimos milisegundos de escrituras; la arquitectura mitiga este riesgo mediante la política de retención local del agente hasta la confirmación explícita de persistencia por el backend.

En el dominio de servicios del backend, la decisión de instancia única implica un único punto de falla con tiempo de recuperación dependiente de los healthchecks de Docker Compose; el patrón de init-container reduce ese tiempo significativamente, y la cola local del agente preserva el servicio durante la ventana de reinicio. En el dominio de notificaciones, la licencia de uso sostenible de n8n prohíbe su reventa o incorporación como componente principal de productos comerciales, restricción compatible con el carácter académico del presente trabajo pero mencionable como consideración para implantaciones productivas; la política de fallbacks automáticos permite operar sin n8n si ello resultara necesario. En el dominio de contraseñas, Argon2id con parámetros fijos puede resultar desmedido o insuficiente según el hardware de despliegue; la parametrización por entorno con benchmark al arranque mitiga este problema.

6.7 Posicionamiento respecto de Falco y Tetragon

Conviene diferenciar la presente propuesta de tecnologías de observabilidad y seguridad en tiempo de ejecución que frecuentemente se mencionan en contextos similares pero que operan en un nivel de abstracción distinto. Falco, proyecto de Sysdig graduado en la Cloud Native Computing Foundation en 2024, y Tetragon, desarrollado por Isovalent y posteriormente por Cisco, son herramientas de seguridad en tiempo de ejecución basadas en el subsistema eBPF del núcleo, cuyo dominio primario es la detección de patrones sospechosos sobre el flujo de llamadas al sistema (Babić et al., 2025). Ambas superan a la presente propuesta en granularidad de detección sobre comportamientos de proceso, conexiones de red y actividad de usuario.

Sin embargo, ninguna de estas herramientas implementa las funciones canónicas de un sistema de Monitoreo de Integridad de Archivos propiamente dicho: no mantienen ni cifran una baseline criptográfica del estado de archivos, no realizan comparación diferencial contra dicha baseline, no modelan una cadena de eventos con resolución de concurrencia para flujos de aprobación humana, no restauran automáticamente desde baseline, y no mantienen un registro de auditoría separado con retención diferenciada. Tales capacidades pueden

construirse encima de Falco o Tetragon mediante desarrollo adicional, pero no vienen provistas por esas plataformas. En consecuencia, la comparación apropiada para la presente propuesta se establece respecto de sistemas FIM específicos como Tripwire, AIDE o Wazuh, no respecto de herramientas de seguridad en tiempo de ejecución. La evaluación de una arquitectura híbrida que delegue la detección de bajo nivel en Tetragon y conserve las capas superiores de la presente propuesta se formula como línea de trabajo futuro en el Capítulo 8.

CAPÍTULO 7. CONCLUSIONES

«La ciencia es una forma de no engañarnos a nosotros mismos. El primer principio es que uno no debe engañarse, y uno es la persona más fácil de engañar.»

— Richard Feynman

El presente capítulo sintetiza las contribuciones del Trabajo Final, evalúa el grado de cumplimiento de los objetivos formulados y extrae las lecciones aprendidas durante el proceso de diseño.

7.1 Cumplimiento de los objetivos formulados

El trabajo cumplió los objetivos específicos de diseño enunciados en el apartado 1.5. Se analizaron los fundamentos teóricos de las funciones hash criptográficas, se caracterizó el estado del arte del bienio 2024-2026, se sistematizó el marco normativo argentino aplicable incluyendo las actualizaciones recientes del Decreto 941/2025, se formuló el modelo de amenazas STRIDE aplicado al propio sistema, se diseñó la arquitectura distribuida con seis componentes desacoplados, se especificó el protocolo de entrega exactamente-una-vez, se diseñaron el agente y el backend con las propiedades requeridas, se especificó la arquitectura criptográfica, se formuló la política de notificaciones con fallbacks, y se diseñó el protocolo experimental reproducible. La implementación efectiva y la ejecución del protocolo sobre el laboratorio definitivo constituyen la fase inmediata siguiente del proyecto.

7.2 Contribución teórica

El trabajo aporta evidencia de diseño a favor de la hipótesis de que las arquitecturas distribuidas basadas en componentes de código abierto del ecosistema Python y JavaScript constituyen una herramienta viable para la construcción de soluciones de seguridad operacional en contextos con recursos limitados. Adicionalmente, documenta formalmente dos contribuciones conceptuales: el esquema de consistencia combinado (cadena superseded junto con protocolo exactamente-una-vez y bloqueo optimista) para flujos de aprobación humana en sistemas FIM, y la composición criptográfica en capas (TLS mutuo con autoridad certificadora propia, HMAC sobre comandos, y cifrado autenticado de baseline con

derivación HKDF) que articula defensa en profundidad con granularidad superior a la habitual en sistemas open source comparables.

7.3 Contribución metodológica

El trabajo aporta un protocolo experimental reproducible con diseño comparativo contra grupo de control, baterías complementarias con umbrales derivados de la literatura, y un modelo STRIDE aplicado al propio sistema. Este último aporte resulta particularmente valioso desde el punto de vista formativo: evidencia que los trabajos de ingeniería académica pueden y deben alcanzar rigor contractual equivalente al de los documentos de diseño industriales.

7.4 Contribución práctica y normativa

El trabajo proporciona un diseño técnico completo y auditado que podrá ser implementado y eventualmente adoptado por organizaciones argentinas una vez completada la fase de implementación. La contribución se amplía mediante la sistematización del marco normativo argentino, lo que facilita a los implementadores establecer la vinculación entre el control técnico y las exigencias de cumplimiento de la Ley 25.326 y su normativa complementaria.

7.5 Contribución formativa

El proyecto permitió articular en un diseño tecnológico cohesivo las competencias adquiridas a lo largo de la Tecnicatura Universitaria en Programación, integrando programación en Python y JavaScript, desarrollo frontend con React y TypeScript, diseño de bases de datos relacionales, protocolos de integración web, mensajería asíncrona, criptografía aplicada, y metodologías de investigación. El enfrentamiento con un problema real de ciberseguridad demandó habilidades no explícitamente cubiertas por el plan de estudios pero características de la práctica profesional: lectura crítica de documentación técnica y estándares internacionales, análisis comparativo con criterios objetivos, elaboración de documentación de decisiones, y comunicación escrita de razonamientos técnicos con rigor académico.

7.6 Reconocimiento de limitaciones y compromiso de adenda

El trabajo presenta limitaciones propias de un estudio de alcance acotado. La más relevante es que, al momento de presentación, la implementación y la ejecución del protocolo no se encuentran concluidas, por lo que los resultados cuantitativos definitivos se incorporarán como adenda una vez ejecutada la fase experimental. Las amenazas a la validez analizadas en el Capítulo 6 no pueden ser completamente eliminadas en el contexto de un trabajo de tecnicatura. No obstante, la calidad y profundidad del diseño documentado permite afirmar que el enfoque propuesto constituye una alternativa genuina para el conjunto de problemas identificados, con potencial de despliegue en contextos de aplicación real una vez completada la implementación.

Los autores asumen el compromiso formal de publicar una adenda metodológica que contenga, como mínimo, lo siguiente: los valores empíricos resultantes de la ejecución íntegra del protocolo del Capítulo 5 completando todas las planillas pendientes; el dictamen final sobre la confirmación o refutación de la hipótesis mediante comparación contra los umbrales del apartado 1.6; las actualizaciones pertinentes a los apartados del Capítulo 6 que quedan supeditados a los resultados empíricos; y la publicación del repositorio de código fuente bajo licencia libre, conforme al compromiso del Anexo F.

CAPÍTULO 8. RECOMENDACIONES Y TRABAJO FUTURO

«Lo importante es que sigas preguntándote qué sucede después.»

— Tradición oral de la ingeniería de sistemas

El desarrollo del trabajo permitió identificar numerosas líneas de extensión que exceden el alcance actual pero resultan prometedoras para futuras iteraciones.

8.1 Completar la fase de implementación y validación

La línea inmediata consiste en implementar los componentes conforme al diseño documentado, ejecutar íntegramente el protocolo experimental sobre el laboratorio definitivo, y completar las planillas de captura del Capítulo 5. Los datos resultantes permitirán verificar o refutar empíricamente la hipótesis y sustentar las conclusiones cuantitativas que actualmente se presentan como esperadas.

8.2 Alta disponibilidad del backend

La decisión de instancia única fija el backend como servicio no replicado en el producto mínimo viable. Para despliegues productivos que exijan alta disponibilidad se propone evaluar arquitecturas multi-réplica con coordinación mediante consumer groups de Valkey, bloqueo optimista en base de datos como método primario de consistencia, y limitación de tasa distribuida. La mayor parte de las decisiones técnicas del presente trabajo resultan compatibles con multi-réplica; otras requieren ajustes puntuales documentables.

8.3 Extensión de la raíz de confianza al núcleo

Se propone la integración con mecanismos de integridad a nivel de núcleo como Integrity Measurement Architecture (IMA), dm-verity o Secure Boot. Estos mecanismos proveen garantías criptográficas sobre los archivos del sistema con independencia del agente de espacio de usuario, y mitigan la amenaza residual de rootkits identificada en el modelo STRIDE.

8.4 Arquitectura híbrida con Tetragon

Se propone evaluar una arquitectura híbrida que delegue la detección de bajo nivel de comportamientos sospechosos en Tetragon sobre eBPF, y conserve las capas superiores de la presente propuesta (baseline cifrada, cadena superseded, flujo de aprobación humana, auditoría con retención ilimitada). Esta combinación capturaría las fortalezas complementarias de ambos enfoques.

8.5 Respuesta automatizada activa

Se propone la extensión hacia capacidades de respuesta activa más allá de la restauración y la cuarentena: aislamiento del anfitrión afectado mediante reglas de firewall dinámicas, restauración desde copias de seguridad verificadas externamente, y apertura automática de tickets en sistemas de gestión de incidentes. Cada acción requiere análisis previo de sus implicancias operativas para evitar denegaciones de servicio auto-infligidas.

8.6 Gestión inteligente de falsos positivos

Se propone un módulo de clasificación basado en aprendizaje automático para distinguir modificaciones legítimas de archivos rotativos o configuraciones actualizadas por procesos autorizados, de modificaciones sospechosas. Complementariamente, listas blancas dinámicas auditadas para excluir del monitoreo archivos cuyo cambio frecuente autorizado genere ruido operativo.

8.7 Integración con ecosistemas SIEM

Se propone construir conectores para enviar eventos FIM hacia plataformas SIEM empleando formatos estándar como Common Event Format, aprovechando la infraestructura de webhooks existente a través de n8n. Esta extensión convertiría al sistema en fuente de telemetría dentro de arquitecturas de seguridad más amplias sin requerir modificaciones al backend.

8.8 Empaquetado nativo y recomendaciones operativas

Para facilitar la adopción institucional se propone empaquetar el agente en formato Debian y RPM, lo que simplifica la gestión del ciclo de vida mediante los gestores de paquetes nativos de cada distribución Linux. Complementariamente, para organizaciones que

adopten el sistema, se formulan recomendaciones operativas: implementar copia de seguridad automatizada de PostgreSQL y de los secretos críticos; monitorear externamente el sistema FIM mediante Prometheus con Grafana consumiendo el endpoint de salud; ejecutar ejercicios regulares de simulación de incidentes; revisar periódicamente las reglas para adaptarlas a la evolución de las aplicaciones; y documentar explícitamente la política de comunicación hacia los usuarios sobre qué se monitorea y con qué propósito, conforme a las exigencias de transparencia del marco normativo argentino.

CAPÍTULO 9. CONSIDERACIONES ÉTICAS

«El ingeniero no es solamente un técnico: es un actor moral cuyas decisiones afectan a terceros que no participan de ellas.»

— Código de Ética del Consejo Profesional de Ciencias Informáticas

El desarrollo de sistemas de Monitoreo de Integridad de Archivos involucra la observación sistemática de actividad sobre archivos que pueden contener información sensible, lo cual plantea consideraciones éticas ineludibles que el presente capítulo explicita.

9.1 Principios orientadores del diseño

El primer principio orientador es la delimitación estricta del alcance del monitoreo hacia archivos del sistema operativo, binarios, bibliotecas compartidas y archivos de configuración. El sistema excluye explícitamente de su ámbito los directorios que contienen datos personales, las carpetas home de usuarios, los archivos de correo electrónico y cualquier repositorio que pueda contener información protegida, salvo razones específicas documentadas y con consentimiento informado de los titulares. Este principio operacionaliza la minimización del alcance propia de la Ley 25.326.

El segundo principio es la minimización de datos almacenados. El sistema registra exclusivamente metadatos necesarios: ruta, hash SHA-256, timestamp y tipo de evento. En ningún caso se almacena contenido de los archivos monitoreados. La política de logs refuerza este principio prohibiendo explícitamente el registro del contenido de las diferencias detectadas; solo se registran los hashes anterior y nuevo junto con la diferencia de tamaño. El tercer principio es la transparencia: la implementación debe acompañarse de política explícita que informe a los usuarios sobre qué se monitorea, con qué propósito, durante cuánto tiempo y quiénes acceden a los registros. El cuarto principio es la proporcionalidad: la decisión de desplegar el sistema debe ser producto de análisis documentado que evalúe, para cada conjunto de archivos, si su criticidad justifica el monitoreo continuo.

9.2 Responsabilidad en la divulgación técnica

El presente trabajo, al hacer públicos los detalles técnicos mediante la descripción detallada de la arquitectura, asume el compromiso ético de contribuir al fortalecimiento

defensivo de la comunidad sin proveer información explotable por adversarios. El Capítulo 2 documenta limitaciones conocidas de fanotify; esta información no constituye una técnica de evasión novedosa sino una limitación documentada en múltiples fuentes previas. Su inclusión alerta a los implementadores sobre la necesidad de defensas complementarias y resulta consistente con el principio de divulgación responsable.

9.3 Uso de herramientas de inteligencia artificial

En concordancia con los lineamientos éticos emergentes sobre el uso de inteligencia artificial en la producción académica, los autores declaran que durante la redacción del presente trabajo se utilizaron herramientas de IA generativa con fines de corrección estilística, verificación de consistencia y organización expositiva. Las siguientes dimensiones son de autoría exclusivamente humana: la identificación del problema de investigación, la formulación de la hipótesis y los objetivos, las decisiones de arquitectura y diseño técnico, la selección del stack tecnológico, la definición del protocolo experimental y la planificación del trabajo futuro. Las contribuciones de las herramientas se limitaron a producción textual bajo supervisión y validación permanente de los autores.

9.4 Adecuación a la normativa argentina

El tratamiento de datos personales de los operadores del sistema se realiza conforme a los principios de la Ley 25.326, incluyendo finalidad legítima, proporcionalidad y mínima retención. La tabla de auditoría con retención ilimitada no almacena datos personales en sentido estricto sino metadatos de acciones administrativas de operadores ya identificados en el contexto de su relación laboral. Los operadores deberán ser informados explícitamente sobre este tratamiento mediante política de privacidad publicada y dispondrán de los mecanismos para ejercer los derechos que la ley les reconoce. Las organizaciones que implementen el sistema en territorio argentino deberán cumplir las obligaciones ante la AAIP cuando resulten aplicables. El desarrollo del presente trabajo se condujo bajo los principios del Código de Ética del Consejo Profesional de Ciencias Informáticas.

CAPÍTULO 10. GLOSARIO

«Las palabras son herramientas cuya forma se ajusta al trabajo que deben realizar.»

— Adaptado de Wittgenstein

El presente glosario recoge los términos técnicos que aparecen a lo largo del trabajo, junto con las leyes argentinas mencionadas, con el propósito de facilitar la lectura a evaluadores cuyo perfil profesional no necesariamente coincida con el de la programación. Las entradas se organizan en dos secciones: la primera, alfabética, comprende términos técnicos; la segunda, por orden de jerarquía normativa y temporal, comprende los instrumentos legales argentinos invocados en la tesis. Las entradas breves ya explicadas mediante siglas en la Lista de Abreviaturas de los preliminares no se repiten aquí a menos que el concepto amerite definición extendida.

10.1 Términos técnicos

AES-256-GCM. Esquema de cifrado simétrico autenticado. Combina el cifrado estándar AES con una clave de 256 bits y el modo de operación Galois/Counter Mode, que provee simultáneamente confidencialidad e integridad en una sola operación. En el presente trabajo se emplea para cifrar la baseline almacenada en el disco del agente.

Agente. Componente de software que se instala en cada anfitrión monitoreado. En esta tesis opera como servicio del sistema gestionado por systemd y observa los eventos de archivo mediante el subsistema fanotify del núcleo Linux.

API. Sigla de Application Programming Interface, interfaz de programación de aplicaciones. Mecanismo formal mediante el cual un componente de software expone sus capacidades para que otros componentes las consuman.

Argon2id. Función de hashing con consumo intensivo de memoria, ganadora de la Password Hashing Competition de 2015. Se emplea para almacenar contraseñas de operadores del sistema de manera que resulte costoso para un atacante intentar descubrirlas incluso disponiendo de la base de datos.

Backend. Parte del sistema que se ejecuta en un servidor y no es visible directamente para el usuario. Contiene la lógica de negocio, gestiona la base de datos y expone una API que el frontend consume.

Baseline. Estado de referencia legítimo de los archivos monitoreados, expresado como una tabla que asocia cada ruta con su huella digital criptográfica SHA-256. Es el patrón contra el cual se comparan las modificaciones futuras para decidir si son legítimas o no.

Bloqueo optimista. Técnica de control de concurrencia en bases de datos que evita bloquear registros para su lectura. En su lugar, cada registro tiene un número de versión, y el intento de actualización falla si la versión cambió desde el último acceso.

CA propia. Autoridad certificadora gestionada internamente por la propia organización, que emite certificados digitales para autenticar a sus agentes sin depender de autoridades comerciales públicas.

CAP_SYS_ADMIN. Capacidad del núcleo Linux que otorga permisos administrativos especializados sin requerir el usuario root. El agente la necesita para operar fanotify en modo completo.

Certificado X.509. Documento digital estandarizado que asocia una clave pública con una identidad (persona, servicio o dispositivo) y está firmado por una autoridad certificadora. Es el formato utilizado por TLS.

Clean Architecture. Patrón de organización de código que aísla la lógica de negocio de las dependencias tecnológicas concretas, organizándolas en capas donde las más externas dependen de las más internas pero no al revés.

Contenedor. Forma ligera de virtualización en la que un conjunto de procesos comparte el núcleo del sistema operativo anfitrión pero se ejecuta con su propio sistema de archivos, redes y vista de procesos aislados.

Docker / Docker Compose. Docker es la plataforma de contenedores más difundida. Docker Compose es una herramienta complementaria que describe en un archivo YAML un conjunto de servicios conformando una aplicación multi-contenedor.

eBPF. Tecnología del núcleo Linux que permite ejecutar pequeños programas seguros dentro del núcleo, con propósitos de observabilidad, redes o seguridad, sin modificar el código del propio núcleo.

fanotify. Subsistema del núcleo Linux, incorporado en 2010, que notifica a procesos de usuario sobre operaciones realizadas en archivos. Sucesor técnico de inotify con

capacidades específicas para sistemas de seguridad, incluyendo la capacidad de bloquear operaciones antes de que ocurran.

FastAPI. Framework de desarrollo web escrito en Python, orientado a la construcción de APIs con alto rendimiento, validación automática de datos y documentación automática conforme al estándar OpenAPI.

FIM. Sigla de File Integrity Monitoring, Monitoreo de Integridad de Archivos. Categoría de herramienta de seguridad cuyo propósito es detectar alteraciones no autorizadas de archivos del sistema comparando su estado actual contra una baseline de referencia.

Frontend. Parte del sistema con la cual el usuario interactúa directamente desde su navegador. En este trabajo se construye con React y TypeScript.

Hash / SHA-256. Función matemática que transforma una entrada de longitud arbitraria en una salida de longitud fija. SHA-256 produce 256 bits (64 caracteres hexadecimales) y es el estándar criptográfico adoptado por el presente trabajo para las huellas digitales de archivos.

HKDF-SHA256. Función de derivación de claves estandarizada en el RFC 5869. Transforma un material secreto inicial en una clave criptográfica de uso específico incorporando una sal y una cadena de contexto.

HMAC-SHA256. Esquema de autenticación de mensajes basado en función hash, que usa SHA-256 como primitiva. Verifica simultáneamente que un mensaje proviene del emisor esperado y que no fue alterado.

inotify. Subsistema más antiguo del núcleo Linux para notificar eventos de archivo. Tiene limitaciones como no ser recursivo de forma nativa y poder perder eventos ante saturación. Sustituido funcionalmente por fanotify.

JSON. Formato de texto estandarizado para representar datos estructurados, de amplio uso en la comunicación entre servicios web.

JWT. Sigla de JSON Web Token. Formato de token de autenticación que contiene información codificada en JSON y firmada criptográficamente. Se emplea para autenticar operadores contra el backend.

Kernel / Núcleo Linux. Componente central del sistema operativo Linux. Gestiona el acceso a los dispositivos físicos, la memoria, los procesos y los sistemas de archivos. Los subsistemas fanotify e inotify son funciones provistas por el núcleo.

- mTLS.** Sigla de mutual TLS, autenticación mutua mediante certificados sobre el protocolo TLS. Tanto cliente como servidor presentan certificados y se verifican recíprocamente.
- n8n.** Plataforma de código abierto para automatizar flujos de trabajo, con un editor visual basado en nodos. En esta tesis se emplea exclusivamente como enrutador de notificaciones hacia canales externos como correo electrónico o mensajería corporativa.
- ORM.** Sigla de Object-Relational Mapper, mapeador objeto-relacional. Biblioteca que permite representar las tablas de una base de datos como clases de un lenguaje de programación, simplificando su manipulación.
- Patrón Repository.** Patrón de diseño que encapsula en clases dedicadas todo el código que accede a la base de datos para cada entidad del dominio, aislándolo del resto del sistema.
- Payload.** Contenido útil de un mensaje, distinguido de los metadatos o el encabezado. En un webhook, el payload es el cuerpo JSON que describe el evento.
- PostgreSQL.** Sistema de gestión de bases de datos relacionales de código abierto, con una larga trayectoria de estabilidad y un conjunto amplio de características avanzadas.
- Python.** Lenguaje de programación de alto nivel, interpretado y de propósito general. Se emplea en este trabajo para el agente y para el backend.
- React.** Biblioteca de JavaScript para construir interfaces de usuario basadas en componentes reutilizables. Desarrollada inicialmente por Facebook y actualmente de código abierto.
- Rootkit.** Tipo de software malicioso que se instala a nivel del núcleo del sistema operativo u otros niveles privilegiados, con el propósito específico de ocultar su propia presencia y la de otras actividades maliciosas.
- SOAR.** Sigla de Security Orchestration, Automation and Response. Categoría de plataformas que orquestan herramientas de seguridad heterogéneas, automatizan tareas repetitivas y gestionan la respuesta a incidentes de forma estructurada.
- systemd.** Gestor de servicios y procesos de inicio adoptado por las principales distribuciones de Linux contemporáneas. El agente del presente sistema se despliega como unidad de systemd, lo que simplifica su arranque automático, reinicio ante fallo y gestión del ciclo de vida.
- TLS 1.3.** Versión vigente del protocolo Transport Layer Security, que reemplaza a TLS 1.2. Provee cifrado y autenticación del canal de comunicación entre dos partes, y es el mecanismo que opera detrás del protocolo HTTPS.

Unit of Work. Patrón de diseño que agrupa múltiples operaciones de acceso a datos bajo una única transacción, garantizando que todas se confirmen en conjunto o ninguna lo haga.

UUID. Sigla de Universally Unique Identifier. Identificador de 128 bits generado de modo tal que la probabilidad de colisión entre dos identificadores creados por separado es prácticamente nula.

Valkey / Valkey Streams. Valkey es un almacén de estructura de datos en memoria, derivado del proyecto Redis en 2024 bajo gobernanza de la Linux Foundation. Sus streams son una estructura de datos persistente y ordenada por tiempo que soporta semántica de mensajería.

Webhook. Mecanismo mediante el cual un sistema notifica a otro la ocurrencia de un evento enviando una solicitud HTTP a una URL preconfigurada. Es el mecanismo principal de integración con n8n en este trabajo.

Zero Trust. Arquitectura de seguridad que rechaza el concepto de perímetro confiable y exige verificación explícita de cada interacción, siguiendo los principios de verificación permanente, privilegio mínimo y asunción de compromiso.

10.2 Instrumentos legales argentinos invocados

Las entradas siguientes identifican la normativa argentina citada en el trabajo, sintetizan su contenido general y explicitan el motivo por el cual el diseño del sistema toma dicho instrumento como referencia o restricción.

Constitución Nacional, artículo 43 (incorporado por la reforma de 1994). Consagra la acción de hábeas data y reconoce el derecho de toda persona a conocer los datos que sobre ella existen en registros públicos o privados destinados a proveer informes, así como a exigir su supresión, rectificación, confidencialidad o actualización en caso de falsedad o discriminación. Es el vértice constitucional del régimen de protección de datos y el fundamento último del marco regulatorio descripto a continuación. Su relevancia para la tesis es que establece el carácter de derecho fundamental de la protección de datos personales, lo cual justifica la exigencia de controles técnicos como el aquí propuesto.

Ley 25.326 de Protección de los Datos Personales (sancionada en 2000, con actualizaciones incorporadas hasta marzo de 2026). Constituye el régimen legal integral para la protección de los datos personales en Argentina. Su artículo 9.º obliga al responsable del tratamiento a adoptar medidas técnicas y organizativas para garantizar la

seguridad y confidencialidad de los datos, y específicamente a evitar su adulteración, pérdida, consulta o tratamiento no autorizado, y a permitir detectar desviaciones. Este artículo constituye el fundamento legal directo de la adopción de sistemas de Monitoreo de Integridad de Archivos en el ordenamiento argentino: la plataforma propuesta implementa una medida técnica orientada a detectar adulteraciones, objetivo que la ley impone como deber del responsable del tratamiento.

Ley 26.388 de Delitos Informáticos (sancionada en 2008). Modifica el Código Penal argentino para tipificar como delitos el acceso ilegítimo a sistemas informáticos, el daño informático (alteración, destrucción o inutilización de datos o programas), la interrupción de comunicaciones electrónicas, y otras conductas del mismo ámbito. En el diseño de la tesis esta ley es relevante porque la tabla de auditoría del sistema, especificada en el Capítulo 4, conserva los registros con retención ilimitada y puede servir como evidencia digital en eventuales investigaciones judiciales conducidas bajo este marco penal.

Resolución AAIP 47/2018 — Medidas de Seguridad Recomendadas. Emitida por la Agencia de Acceso a la Información Pública, aprueba un conjunto de medidas de seguridad recomendadas para el tratamiento de datos personales en medios informatizados, organizadas en cinco procesos: recolección, control de acceso, conservación, destrucción y gestión de incidentes. En el diseño de la tesis, el sistema propuesto contribuye específicamente al proceso de control de acceso mediante el registro auditable de todas las acciones administrativas sobre archivos, y al proceso de gestión de incidentes mediante detección temprana y notificación automatizada. Esta resolución justifica directamente la decisión arquitectónica de separar la tabla de auditoría de los logs estructurados y de mantenerla con retención ilimitada.

Resolución AAIP 126/2024 — Régimen Sancionatorio. Actualiza el régimen sancionatorio bajo la Ley 25.326, estableciendo una clasificación tripartita de las infracciones en leves, graves y muy graves, y fijando los criterios de graduación de las sanciones. Es especialmente relevante para la tesis que el propio incumplimiento del deber de informar las medidas de seguridad implementadas figura entre las infracciones leves; esto refuerza la importancia de contar con documentación técnica auditable como la producida por la plataforma propuesta, que deja trazabilidad completa de los controles aplicados.

Resolución 44/2023 — Segunda Estrategia Nacional de Ciberseguridad. Emitida por la Secretaría de Innovación Pública, aprueba la Segunda Estrategia Nacional de Ciberseguridad. Los objetivos centrales incluyen proteger las infraestructuras críticas nacionales, fortalecer las capacidades de prevención, detección y respuesta a incidentes, y

promover la formación de capital humano especializado en ciberseguridad. El diseño de la tesis se alinea directamente con el objetivo de detección y respuesta, en tanto la plataforma detecta modificaciones en tiempo real y notifica para habilitar la respuesta operativa.

Decisión Administrativa 641/2021 — Requisitos Mínimos de Seguridad de la Información. Emitida por la Jefatura de Gabinete de Ministros, aprueba los Requisitos Mínimos de Seguridad de la Información para los organismos del Sector Público Nacional. Establece controles técnicos obligatorios: gestión de identidades, cifrado, respaldos, registros de auditoría y gestión de incidentes. La tesis toma esta normativa como referencia para enmarcar los controles técnicos del sistema propuesto, que cubre especialmente los requerimientos asociados a detección de alteraciones, registros de auditoría, cifrado y gestión de incidentes.

Decreto de Necesidad y Urgencia 941/2025 — Creación del Centro Nacional de Ciberseguridad. Publicado el 2 de enero de 2026, crea el Centro Nacional de Ciberseguridad (CNC) como organismo descentralizado en la órbita de la Secretaría de Innovación, Ciencia y Tecnología de la Jefatura de Gabinete de Ministros. El CNC queda constituido como autoridad nacional en materia de ciberseguridad, con competencia sobre la protección de infraestructuras críticas, el monitoreo y respuesta ante incidentes del Sector Público Nacional y la conducción del CERT nacional. El decreto separa formalmente las funciones de ciberseguridad de las de ciberinteligencia. Su relevancia para la tesis radica en consolidar la autoridad técnica nacional ante la cual podrían eventualmente reportarse incidentes detectados por sistemas como el aquí propuesto.

Decreto 92/2026 — Designación de Autoridades del CNC. Publicado el 9 de febrero de 2026, designó formalmente al doctor Ariel Waissbein como Director Ejecutivo del Centro Nacional de Ciberseguridad y al señor Ezequiel Gutesman como Subdirector Ejecutivo, dando cumplimiento al artículo 25 del Decreto 941/2025 y activando operativamente el organismo.

CAPÍTULO 11. REFERENCIAS BIBLIOGRÁFICAS

«Nos erguimos sobre los hombros de gigantes.»

— Atribuida a Isaac Newton

Las referencias se presentan conforme al formato APA 7.^a edición, organizadas en seis secciones temáticas. La selección privilegia publicaciones de los últimos dos años para las fuentes que describen el estado del arte y los reportes de amenazas, y se apoya en estándares y normativa para los fundamentos conceptuales, con incorporación de textos clásicos para los aspectos metodológicos y criptográficos de más larga vigencia.

11.1 Normativa argentina

Agencia de Acceso a la Información Pública. (2018). Resolución 47/2018: Medidas de Seguridad Recomendadas para el Tratamiento y Conservación de los Datos Personales en Medios Informatizados. Boletín Oficial de la República Argentina. <https://www.argentina.gob.ar/normativa/nacional/resolución-47-2018-312662>

Agencia de Acceso a la Información Pública. (2024). Resolución 126/2024: Clasificación de las Conductas Punibles y Graduación de las Sanciones bajo la Ley 25.326. Boletín Oficial de la República Argentina.

Honorable Congreso de la Nación Argentina. (2000). Ley 25.326 de Protección de los Datos Personales (texto actualizado al 12 de marzo de 2026). Boletín Oficial de la República Argentina. <https://www.argentina.gob.ar/normativa/nacional/ley-25326-64790>

Honorable Congreso de la Nación Argentina. (2008). Ley 26.388 de Delitos Informáticos. Boletín Oficial de la República Argentina.

Jefatura de Gabinete de Ministros. (2021). Decisión Administrativa 641/2021: Requisitos Mínimos de Seguridad de la Información para Organismos del Sector Público Nacional. Boletín Oficial de la República Argentina.

Poder Ejecutivo Nacional. (2025). Decreto de Necesidad y Urgencia 941/2025: Creación del Centro Nacional de Ciberseguridad. Boletín Oficial de la República Argentina, 2 de enero de 2026. <https://www.boletinoficial.gob.ar/detalleAviso/primera/337032/20260102>

Poder Ejecutivo Nacional. (2026). Decreto 92/2026: Designación de autoridades del Centro Nacional de Ciberseguridad. Boletín Oficial de la República Argentina, 9 de febrero de 2026.

Secretaría de Innovación Pública. (2023). Resolución 44/2023: Segunda Estrategia Nacional de Ciberseguridad. Boletín Oficial de la República Argentina.

11.2 Estándares internacionales

Biryukov, A., Dinu, D., Khovratovich, D., & Josefsson, S. (2021). Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications (RFC 9106). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9106>

Cichonski, P., Millar, T., Grance, T., & Scarfone, K. (2012). Computer Security Incident Handling Guide (NIST Special Publication 800-61 Revision 2). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-61r2>

Krawczyk, H., & Eronen, P. (2010). HMAC-based Extract-and-Expand Key Derivation Function (HKDF) (RFC 5869). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc5869>

National Institute of Standards and Technology. (2015). FIPS PUB 180-4: Secure Hash Standard (SHS). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.180-4>

Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3 (RFC 8446). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8446>

Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). Zero Trust Architecture (NIST Special Publication 800-207). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-207>

11.3 Reportes de amenazas y estado de la industria (2024-2025)

IBM Security. (2024). X-Force Threat Intelligence Index 2024. IBM Corporation.

Mandiant. (2024). M-Trends 2024: Special Report on the State of the Cyber-Threat Landscape. Google Cloud / Mandiant.

Verizon Business. (2024). 2024 Data Breach Investigations Report (DBIR). Verizon.

11.4 Publicaciones académicas recientes

- Babić, K., Shu, X., & Li, J. (2025). Runtime Security Observability in Linux: A Comparative Analysis of eBPF-based Tools. *IEEE Security & Privacy*, 23(3), 42-51.
- Clément, R., Duval, P., & Menéndez, H. (2025). Supply-Chain Backdoor in Open-Source Compression: A Forensic Reconstruction of CVE-2024-3094 (XZ Utils). *ACM Transactions on Privacy and Security*, 28(1), Article 5. <https://doi.org/10.1145/3712345>
- Leurent, G., & Peyrin, T. (2020). SHA-1 is a Shambles: First Chosen-Prefix Collision on SHA-1 and Application to the PGP Web of Trust. En *Proceedings of the 29th USENIX Security Symposium* (pp. 1839-1856). USENIX Association.

11.5 Libros y manuales metodológicos

- Hernández Sampieri, R., Fernández Collado, C., & Baptista Lucio, P. (2014). *Metodología de la Investigación* (6.ª ed.). McGraw-Hill Interamericana.
- Menezes, A. J., van Oorschot, P. C., & Vanstone, S. A. (1996). *Handbook of Applied Cryptography*. CRC Press.
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer.

11.6 Documentación técnica oficial

- FastAPI. (2025). FastAPI Documentation. <https://fastapi.tiangolo.com/>
- Kernel Linux. (2025). fanotify(7) — Monitoring filesystem events. <https://man7.org/linux/man-pages/man7/fanotify.7.html>
- n8n Documentation. (2025). n8n: Workflow automation platform. <https://docs.n8n.io/>
- Pyfanotify Contributors. (2024). pyfanotify 0.3.0: Linux fanotify Python wrapper. PyPI. <https://pypi.org/project/pyfanotify/>
- PostgreSQL Global Development Group. (2025). PostgreSQL 18 Documentation. <https://www.postgresql.org/docs/18/>
- Python Software Foundation. (2025). Python 3.12 Documentation: hashlib module. <https://docs.python.org/3/library/hashlib.html>

Valkey Contributors. (2026). Valkey Documentation. <https://valkey.io/documentation/>

Wazuh Inc. (2025). Wazuh: Open Source Security Platform — Official Documentation. <https://documentation.wazuh.com/>

CAPÍTULO 12. ANEXOS

«El detalle es donde se aloja la verdad, y donde también se esconden los errores.»

— Adaptado de la tradición editorial académica

Los siguientes anexos amplían aspectos específicos de la documentación técnica presentada en el Capítulo 4, con énfasis en los flujos de procesamiento y en extractos representativos de configuraciones centrales.

Anexo A. Flujo de procesamiento del agente

El presente anexo describe en detalle secuencial el flujo operativo del agente, complementando la exposición del apartado 4.3 con granularidad adicional sobre los puntos de sincronización. El flujo se organiza en tres fases temporales bien diferenciadas, coincidentes con las presentadas gráficamente en la Figura 3.

Fase de arranque. El agente, al iniciarse como servicio de systemd, ejecuta secuencialmente las siguientes tareas: lectura del archivo de configuración local y de las variables de entorno; carga del secreto maestro de baseline desde el archivo protegido con permisos restrictivos; derivación de la clave de cifrado simétrico mediante HKDF-SHA256 a partir del secreto maestro, del identificador del agente y de la cadena de propósito; apertura de la baseline cifrada; rehidratación del registro de acciones previas, con procesamiento diferenciado de las entradas pendientes (que se reintentarán) y las entradas en ejecución (que se reportarán al backend como estado indeterminado); verificación de la vigencia del certificado TLS local; establecimiento de la conexión TLS mutua con el backend; consumo del stream commands para recibir configuración actualizada y nueva versión del ruleset; y finalmente configuración de fanotify con las rutas configuradas mediante marcas específicas del tipo de operación a monitorear.

Fase de operación. Por cada evento recibido desde fanotify, el agente ejecuta: cálculo del hash SHA-256 del archivo modificado; consulta a la baseline para obtener el hash de referencia; decisión de descarte silencioso si los hashes coinciden; evaluación del motor de decisión local contra las reglas cacheadas si difieren; escritura de una entrada de registro de acción previa con estado pendiente antes de ejecutar cualquier acción modificatoria; ejecución

de la acción seleccionada entre las cuatro posibles; actualización del registro a completado o fallido con los detalles del resultado; y publicación del evento en el stream events de Valkey con los campos requeridos (identificador UUID, path, hash, estado, detected_at, versión del esquema de mensajes). Si el backend se encuentra indisponible, el evento se serializa al directorio de cola local con política drop-oldest al superar el umbral de cien megabytes.

Fase de sincronización periódica. De modo continuo y paralelo al flujo principal, el agente emite latidos al stream agent_heartbeat cada diez segundos y consume del stream commands los mensajes de confirmación (event_ack) para eliminar las entradas correspondientes de la cola local. Al recibir la señal SIGTERM, transita a modo de drenaje ordenado según el protocolo descrito en el apartado 4.3.

Anexo B. Flujo de aprobación humana en el backend

El flujo de aprobación humana recorre los siguientes pasos cuando el operador interactúa con un evento en estado pending. El frontend carga la lista de eventos mediante el endpoint correspondiente, que retorna cada evento con su versión actual. Cuando el operador selecciona un evento y ejecuta la acción de aprobación, el frontend envía una solicitud al backend incluyendo el identificador del evento y el número de versión que se muestra actualmente. El backend recibe la solicitud en el router correspondiente, autentica al operador mediante el token JWT, invoca el caso de uso ApproveEvent y, dentro del contexto Unit of Work, intenta actualizar el estado del evento ejecutando la sentencia SQL UPDATE con la cláusula WHERE que verifica simultáneamente el identificador, la versión esperada y el estado pending.

Si el UPDATE afecta una fila, el caso de uso ejecuta además la actualización de la baseline con el hash presente en el filesystem, el registro en la tabla de auditoría, la emisión del comando event_ack al agente mediante el stream commands, y el disparo de la notificación mediante el NotificationDispatcher. Todas estas operaciones ocurren dentro de la misma transacción del Unit of Work, que se confirma atómicamente al finalizar el caso de uso. Si alguna operación intermedia falla, la transacción se revierte íntegramente y se retorna error al frontend.

Si el UPDATE afecta cero filas (lo que ocurre cuando otra operación alteró simultáneamente el estado o la versión del evento), el caso de uso retorna HTTP 409 con

cuerpo que identifica el conflicto. El frontend traduce esta respuesta en un mensaje al operador y refresca automáticamente la lista, mostrando el nuevo estado del evento.

Anexo C. Extracto del esquema de persistencia

El presente anexo describe en lenguaje llano el contenido principal de las tablas del esquema PostgreSQL, complementando el modelo entidad-relación presentado como Figura 2. No se incluye el script SQL completo por razones de extensión; los fragmentos relevantes se integran como referencia conceptual.

La tabla `events` contiene el registro temporal de todas las modificaciones detectadas. Cada fila representa una detección individual con identificador UUID único, identificador del agente, ruta del archivo monitoreado, tipo de operación (`create`, `modify`, `delete`), hash SHA-256 anterior y actual, timestamp de detección, timestamp de recepción, estado del evento (`pending`, `approved`, `rejected`, `superseded`, `auto_restored`, `quarantined`, `alert_only`), número de versión para bloqueo optimista, referencia al evento padre en caso de cadena `superseded`, y los identificadores del operador y el timestamp en caso de resolución humana.

La tabla `baseline` mantiene el estado autoritativo por ruta monitoreada: ruta, agente, estado (`present` o `absent`), hash esperado si aplica, timestamp de última modificación y timestamp de última verificación. La tabla `audit_log`, con retención ilimitada, registra cada acción administrativa con identificador UUID, tipo de acción, operador, timestamp, dirección IP de origen, y datos adicionales específicos del tipo de acción. Las tablas auxiliares `failed_notifications`, `rejected_events_audit`, `revoked_certificates` y `ruleset_version` completan el esquema.

Anexo D. Estructura del archivo Docker Compose

El archivo `docker-compose.yml` del servidor central define los siguientes servicios. El servicio `db` ejecuta la imagen oficial de PostgreSQL 18 con volumen persistente para los datos, `healthcheck` mediante la utilidad `pg_isready`, política de reinicio `always`, red interna exclusiva con el backend, y variables de entorno que contienen las credenciales iniciales. El servicio `valkey` ejecuta la imagen oficial de Valkey 9.0 con volumen persistente para los streams, `healthcheck` con el comando `PING`, y política de reinicio `always`.

El servicio db-init ejecuta una única vez al arranque del stack: se monta con el mismo código fuente del backend, depende del servicio db en estado healthy, ejecuta un script que invoca SQLModel para crear las tablas y para insertar el usuario administrador inicial con flag de cambio forzado de contraseña, y finaliza. El servicio backend ejecuta la imagen construida del backend, depende de db-init en estado completed y de valkey en estado healthy, expone el puerto de la API, monta los certificados TLS como secretos de solo lectura, e incluye healthcheck sobre el endpoint /healthz.

El servicio frontend ejecuta la imagen construida del frontend con nginx como servidor web, expone el puerto correspondiente, depende del backend y lleva las cabeceras de hardening configuradas en su propia configuración de nginx. El servicio n8n ejecuta la imagen oficial con volumen persistente para sus flujos, expone su puerto administrativo restringido a la red interna, y se conecta a la misma red que el backend para recibir los webhooks.

Anexo E. Unit file de systemd del agente

El agente se despliega mediante un archivo de unidad de systemd ubicado en /etc/systemd/system/fim-agent.service. La configuración especifica los siguientes aspectos principales. El servicio ejecuta el entrypoint del agente con Python desde el entorno virtual /opt/fim-agent, bajo el usuario de sistema fim-agent, y se reinicia automáticamente ante fallo con una ventana acotada. Se incorporan directivas de hardening propias de systemd: ProtectSystem=strict restringe el sistema de archivos a solo lectura excepto directorios explícitamente permitidos; ProtectHome=yes oculta los directorios home; MemoryDenyWriteExecute=yes prohíbe páginas de memoria simultáneamente escribibles y ejecutables; CapabilityBoundingSet restringe la lista de capacidades del núcleo a únicamente CAP_SYS_ADMIN que se requiere para fanotify; y se establecen límites de memoria (MemoryMax=512M) y cuota de CPU (CPUQuota=50%) para evitar que el agente consuma recursos desmedidos en caso de comportamiento anómalo.

Anexo F. Disponibilidad del código y compromiso de adenda

El código fuente definitivo del sistema, junto con los archivos de configuración, los flujos de n8n exportables, el esquema SQL completo, los scripts de instalación del agente, y el protocolo experimental ejecutable, se publicarán en un repositorio público alojado en la

plataforma GitHub una vez completada la fase de implementación. Hasta ese momento, el documento de arquitectura y los artefactos de diseño constituyen la fuente primaria de información técnica sobre el sistema.

Los autores asumen formalmente los siguientes compromisos: publicar el repositorio bajo una licencia de software libre compatible con los componentes de base; publicar los resultados numéricos completos derivados de la ejecución del protocolo experimental del Capítulo 5; publicar una adenda metodológica que complete las planillas pendientes y que dictamine sobre la confirmación o refutación de la hipótesis; y mantener el repositorio disponible por un plazo no inferior a dos años posteriores a la defensa del trabajo.